

ECE 5760 Lab 3: Multiprocessor Drum Synthesis

Md Sharear Haidar Shad, Michael Joseph Grillo, Stephen John Barlett

March 27, 2026

1 Introduction

In this lab, we developed and implemented a hardware-accelerated drum synthesizer on the DE1-SoC by simulating a damped 2D wave equation on a mesh with discretized nodes. The system computes the node positions over a range of time, where the mesh acts as a vibrating membrane that represents the displacement of the surface of a drum. More specifically, each of our nodes represented the displacement of the drum surface at a specific spacial location. As a result of the iterative amplitude updates, the system is able to generate realistic drum sounds. The amplitude is sampled at an audio rate of 48kHz, allowing for all frequencies audible to humans to be sampled according to the Nyquist rate [3]. The amplitudes are then transmitted to an audio DAC and connected with a speaker to produce continuous real-time sound input.

This system is based on the underlying physics model of a discretized 2D wave equation, which equates a node's amplitude as a function of the surrounding nodes, providing information on the mesh curvature and the time difference between time steps, representing the acceleration of the node. Specifically, we use the values of the node's 4 nearest neighbors to calculate an updated position. Additionally, a damping term is added to allow for accurate modeling of energy transfer out of the system due to forces such as membrane friction and air resistance. Additionally, a nonlinear tension term, ρ is incorporated into the system to model the classic pitch-glide property of a drum, allowing the simulation to more accurately represent physical behavior.

The computation was implemented entirely in Verilog and synthesized onto an FPGA for accelerated calculation. 1.17 signed fixed-point arithmetic was used to efficiently perform required additions, multiplications, and subtractions. The 18 bit size allowed us to fit two multiplications (therefore two columns worth of compute) into one DSP block. This increased the hardware ceiling for this design and allowed us to synthesize more than 87 columns. To maximize the resource utilization in an attempt to create the largest drum possible, we developed a parallel architecture where multiple columns operate in parallel. Each column contains its own compute unit and control logic, which allows for all of the columns to operate in parallel while they iterate through rows sequentially. The state of the nodes are stored in on-chip M10K memory blocks, enabling fast and deterministic access to past and current timestep values across the columns.

Unlike our previous labs that relied on the Hard Processor System (HPS) for control and visualization, the computation and interfacing for this lab lies entirely within the FPGA.

The FPGA manages input generation through switches and buttons, computation through LUTs, memory blocks, and DSP blocks, and audio output, while the HPS is used only for baseline performance comparison and not used in our real-time drum simulation. The computed center-node amplitude is streamed to the onboard audio subsystem to allow for synchronization with the audio codac.

A primary objective of this lab was to maximize the size of the simulated drum by efficiently utilizing FPGA resources such as DSP blocks, on-chip memory, and an FSM that could allow for quick processing of rows. By minimizing the number of DSP blocks used for multiplies for each compute unit and by carefully structuring our patterns of memory access, we were able to scale the drum to a grid size of 128 columns by 100 rows (a total of 12,800 nodes), while maintaining real time performance at 48kHz. Through this approach, our system demonstrates the effectiveness for FPGA based hardware acceleration to solve partial differential equations in real time through discretized optimizations.

Overall, our implementation successfully achieves real-time drum synthesis for large (up to 12,800 node) meshes, maintaining 48kHz audio rate, with efficient hardware utilization, using a parallel, scalable infrastructure, demonstrating the power of FPGAs for parallel, high-throughput numerical simulations.

2 Mathematical Background

The sound produced by a drum can be modeled from the vibrations of the drum membrane as a continuous system described by the 2D wave equation with damping. This equation describes the propagation of disturbances across a 2D sheet and the subsequent transfer of energy as a wave and considers damping forces within the membrane. The continuous damped 2D wave equation can be written as $\frac{1}{c^2} \frac{\partial^2 u}{\partial t^2} + \frac{\partial^2 u}{\partial y^2} - \eta \frac{\partial^2 u}{\partial x^2}$, where c represents the speed of propagation within the medium of the drum membrane, η represents the damping coefficient and $u(x, y, t)$ represents the membrane displacement.

This system can be thought of as a network of mass-spring damping nodes, where a change in one node will cause a change to that node's neighbors. The second spacial derivatives, $\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$ form a Laplacian operator, which measures the curvature of the surface. Thus, when a region has high total curvature, there is a higher restoring force to attempt to 'flatten' such curve than when there is lower total curvature. The second derivative with respect to time, $\frac{\partial^2 u}{\partial t^2}$, represents the acceleration of the membrane, ensuring that the change in the membrane does not only depend on the current position, but on prior momentum. This captures kinetic energy in the system, and tied with the potential energy captured via the Laplacian, total conservation of energy can be maintained, and the oscillatory behavior of the wave can be intuited.

The damping term, $\eta \frac{\partial^2 u}{\partial x^2}$ provides energy dissipation from the system into the environment, allowing for a realistic system model, considering the effect of air resistance in the membrane vibration and the introduction of internal friction within the model. Without such term, our drum would remain at a constant volume for all time, which would not represent realistic function, so including it allows for our system to return to an equilibrium state.

The coefficient c represents the speed of wave transmission across the membrane. This

coefficient c depends on the surface tension τ and surface density σ within the membrane, being defined as $c = \sqrt{\tau/\sigma}$ as described in [?].

2.1 Wave Equation Discretization

In order to simulate the wave equation on hardware, it is imperative to discretize both time and space for the 2D wave equation. As described in the Discretization page of the course website [1] To discretize the wave equation, we can represent the first derivative of $u(t)$ as

$$\frac{d}{dt}u(t) = \frac{u_{i,j}^{n+1} - u_{i,j}^n}{\Delta t} \quad (1)$$

and the second derivative as

$$\frac{d^2}{dt^2}u(t) = \frac{u_{i,j}^{n+1} - 2u_{i,j}^n + u_{i,j}^{n-1}}{\Delta t^2} \quad (2)$$

When this is substituted into the continuous wave equation shown below:

$$\frac{1}{c^2} \frac{\partial^2 u}{\partial t^2} = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - \eta \frac{\partial^2 u}{\partial x^2} \quad (3)$$

we are able to derive a difference approximation for the 2D wave equation as shown below and detailed in the course webpage [1] as:

$$\frac{1}{c^2} \left[\frac{u_{i,j}^{n+1} - 2u_{i,j}^n + u_{i,j}^{n-1}}{\Delta t^2} \right] = \frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} + \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2} - \eta \left(\frac{u_{i,j}^{n+1} - u_{i,j}^{n-1}}{2\Delta t} \right)$$

Assuming that $\Delta x = \Delta y$, this expression can be rearranged into the explicit update form represented by

$$\left(1 + \frac{\eta \Delta t}{2} \right)^{-1} \left\{ \rho (u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n) + 2u_{i,j}^n - \left(1 - \frac{\eta \Delta t}{2} \right) u_{i,j}^{n-1} \right\} \quad (4)$$

where

$$\rho = \left(\frac{c \Delta t}{\Delta x} \right)^2 \quad (5)$$

and ρ is less than 0.5, in order to stay numerically stable. However, critical to the sound of a drum is its "pitch-glide" feature, which occurs when ρ becomes non-linear. The form of enacting pitch-glide in this lab consists of

$$\rho_{\text{eff}} = \min(0.49, \rho_0 + (u_{\text{center}} \cdot G_{\text{tension}})^2)$$

Overall, ρ represents the propagation speed of our drum, where we consider the physical wave speed c , the update time, Δt , and the spacing between each of the nodes, Δx to understand how far a wave will travel in one timestep. A lower ρ corresponds with slower wave propagation and a deeper sound and a higher value for ρ corresponds to a higher

frequency as we have faster wave propagation. When we use ρ_{eff} , we continuously recalculate this speed to change depending on the displacement of the membrane, allowing for further representation of our sound, similar to a real drum.

2.2 FPGA Implementation

In order to attempt to synthesize as many nodes onto our drum as possible, and maximize our hardware usage on the FPGA with optimized calculations, it is possible to simplify the discretized wave equation to reduce the number of multiplies, and eliminate the division in the discretized wave equation.

In our original equation,

$$u_{i,j}^{n+1} = (1 + \eta\Delta t^2)^{-1} \{ \rho (u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j+1}^n + u_{i,j-1}^n - 4u_{i,j}^n) + 2u_{i,j}^n - (1 - \eta\Delta t^2) u_{i,j}^{n-1} \}$$

the term $(1 + \eta\Delta t^2)^{-1}$ currently needs a division to calculate the inverse of $(1 + \eta\Delta t^2)$; however we can use a Taylor approximation to eliminate this division. This approximation is proven as follows:

let

$$x = \eta\Delta t^2$$

so that

$$(1 + \eta\Delta t^2)^{-1} = (1 + x)^{-1}$$

The first-order Taylor expansion of $f(x) = (1 + x)^{-1}$ about $x = 0$ is

$$f(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \dots$$

Since

$$f(0) = 1$$

and

$$f'(x) = -\frac{1}{(1+x)^2} \quad f'(0) = -1$$

the first-order Taylor approximation is

$$\frac{1}{1+x} \approx 1 - x$$

$$f(x) \approx f(0) + f'(0)x$$

with

$$f(0) = 1, \quad f'(x) = -(1+x)^{-2}, \quad f'(0) = -1$$

gives

$$(1+x)^{-1} \approx 1 - x$$

and therefore

$$(1 + \eta\Delta t^2)^{-1} \approx 1 - \eta\Delta t^2$$

Because of this, the division by $(1 + \eta\Delta t^2)$ is replaced with a multiplication by $(1 - \eta\Delta t^2)$, converting the node update into a form that can be implemented using only additions,

multiplications, and shifts, which involves significantly less compute on the FPGA. Since $\eta\Delta t^2$ is small, this approximation stays reasonable for our implementation.

Additionally, our original discretized wave equation appears to use multiple DSP blocks per iterator through a series of multipliers; however, it is possible to reduce this to the use of only one multiplier. The first step in this process is figuring out which multiplies are by a factor of $2^n \forall \in \mathbb{Z}$, and rather than multiplying, bit shifting by $\log_2(x)$, which takes almost no hardware resources.

To multiply by ρ , in order to use only one DSP block, the Laplacian was taken where we considered

$$u_{intermediate} = u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j+1}^n + u_{i,j-1}^n - 4u_{i,j}^n$$

and $u_{intermediate}$ would be multiplied by ρ as $\rho \cdot u_{intermediate}$ to give the scaled Laplacian.

After finding this scaled Laplacian, it appears as if it is necessary to multiply each term in the equation with $1 - \eta\Delta t^2$, with the possible appearance of

$$\left(1 - \frac{\eta\Delta t}{2}\right) \left\{ \rho (u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n) + 2u_{i,j}^n - \left(1 - \frac{\eta\Delta t}{2}\right) u_{i,j}^{n-1} \right\} \quad (6)$$

as

$$u_{i,j}^{n+1} = \left(1 - \frac{\eta\Delta t}{2}\right) [\rho (u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n)] + \left(1 - \frac{\eta\Delta t}{2}\right) 2u_{i,j}^n - (1 - \eta(\Delta t)^2) u_{i,j}^{n-1} \quad (7)$$

However, considering that $\frac{\eta\Delta t}{2}$ is a value of $\approx 2^{-9} - 2^{-10}$, the terms can be factored as

$$\begin{aligned} u_{i,j}^{n+1} = & \left[\rho (u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n) + 2u_{i,j}^n - u_{i,j}^{n-1} \right] \\ & - \frac{\eta\Delta t}{2} \left[\rho (u_{i+1,j}^n + u_{i-1,j}^n + u_{i,j-1}^n + u_{i,j+1}^n - 4u_{i,j}^n) + 2u_{i,j}^n \right] \\ & + \frac{\eta\Delta t}{2} [2\Delta t u_{i,j}^{n-1}] \end{aligned} \quad (8)$$

and since $(1 - \eta(\Delta t)^2)$ can be represented as bitshifts, this form reduces the number of possible multiplications necessary for each iteration to only one, which calculates the value of ρ multiplied by the discrete Laplacian for the wave equation.

Overall, these mathematical considerations make it feasible to calculate the wave equation via FPGA using a variety of factorizations and approximations, allowing for high performance computing of the wave equation to synthesize a drum.

3 Roles of the ARM (HPS) and FPGA

The FPGA does all of the configuration and computation in our complete Drum Synth Design. Our user interface is described in Section 9.2, but essentially we read the state of several switches and buttons to handle the initial conditions of our drum. Based on these initial conditions, when KEY0 is pressed our FPGA FSM sets the initial amplitude of every node and then begins computing the wave equation every $20\mu s$ and sending the audio data

to the audio core via the Audio Bus Master described in Section 10.3. We also handle timing via toggling an IO pin during FSM execution and probing it with an oscilloscope.

The HPS was not used in this lab beyond running the software 30x30 drum. This involved compiling Bruce’s C code for the 30x30 drum, and the Audio Control from HPS demo Quartus project from the course webpage. We made some small modifications to this C program to make it print timing diagnostics (see Section 10.1).

3.1 System-Level Integration

For this lab, our Avalon bus architecture is intentionally streamlined, containing only the components necessary for audio generation and playback. The three devices of interest on the bus are the `Audio_Subsystem`, the `AV_Config`, and the `EBAB_Bus_Master_Audio`. The EBAB serves as the Avalon bus master through which our design communicates with the Audio Core Subsystem and, by extension, the audio codec onboard the DE1-SoC. This bus master exposes six signals to our top-level design, which are used to manipulate the audio slave and are enumerated in Table 6. Our FSM interacts with the audio subsystem through the EBAB’s external bus-master interface using a two-step read-then-write protocol each output cycle. First, the FSM reads the FIFO-space register to determine whether the audio core has sufficient room to accept a new sample. If space is available, the FSM writes the freshly computed audio sample to both the left and right channel data registers, which enqueue the sample into the audio core’s output FIFOs. The audio subsystem then drains these FIFOs at the codec’s playback rate (configured to be 48KHz) and transmits the samples to the DE1-SoC audio codec for analog output. This handshaking approach ensures that our FSM never writes to a full FIFO, preventing sample corruption or dropped data. Because audio output is the only data path in our design, this simple polling mechanism is sufficient — no DMA or interrupt-driven transfer is required.

4 RTL Design

In our design we implemented a hardware accelerator for simulating a 2D vibrating drum using a finite-difference approximation of the wave equation on the Altera DE1-SoC. Our system calculates the time evolution of a discretized grid of nodes, where each of the nodes represent the amplitude of the drum membrane at a specific position. The update equation is derived from the 2D wave equation and takes the form

$$u_{\text{next}} = \rho(u_{\text{left}} + u_{\text{right}} + u_{\text{above}} + u_{\text{below}} - 4u_{\text{center}}) + 2u_{\text{center}} - u_{\text{center_p}},$$

where each node relies on each of its four neighboring nodes and its own values from the current and previous time steps. All values in the system are demonstrated using 1.17 signed fixed-point arithmetic, which means each value uses 18 bits total: 1 sign bit, 1 integer bit, and 16 fractional bits. This type of representation allows values to span about the range $[-2, 2)$ with a resolution of 2^{-16} .

This range is reasonable for the drum simulation because the node amplitudes are usually initialized to very small values (on the order of 1/8 full scale) and they stay bounded over

time due to the stability constraint of $\rho < 0.5$. This means, overflow is prevented while still keeping sufficient precision to accurately capture the oscillatory behavior of this system.

Furthermore, since we are using fixed-point arithmetic instead of floating-point significantly we are reducing hardware complexity, as it allows multiplications to be mapped directly onto FPGA DSP blocks and enables additions and shifts to be implemented with simple logic. This allows lower resource usage, higher clock frequencies, and better scalability when instantiating many compute units in parallel.

The datapath is designed so that each clock cycle computes one node update when the pipeline is filled. The design stores the current and previous values of each node in on-chip M10K memory blocks and updates these values iteratively. Then damping term is applied using bit-shift operations to approximate exponential decay, ensuring that the system is stable over time. The boundary conditions are enforced by making edge node values to zero, which creates a fixed drum boundary.

4.1 Single Node Validation

Before we had scaled to a full two-dimensional grid, we had to first validate the correctness of the finite-difference update equation in isolation. We simulated a single `wave_node` module in ModelSim with all four neighbor inputs (`u_above`, `u_below`, `u_left`, `u_right`) set and held at zero, setting Dirichlet boundary conditions on every face of the node. This change collapses the 2D spatial Laplacian, which usually sums the differences between a node and all four of its neighbors to the expression $-4u_{\text{center}}$, reducing the full 2D wave equation to a one-dimensional damped harmonic oscillator whose analytical behavior is well understood. We decided for this as our entry point because any deviation from the expected sinusoidal decay would unambiguously point to a bug in the arithmetic pipeline rather than in the memory system or control logic, neither of which had been implemented at this stage.

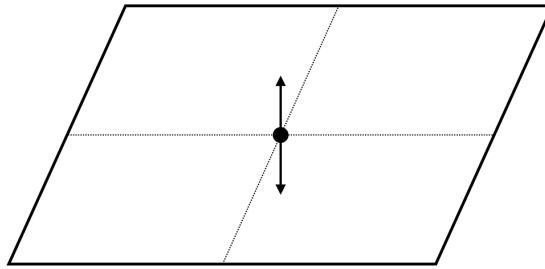


Figure 1: Single node simulation with zero boundary conditions (Dirichlet boundaries on all sides), adapted from [1].

4.1.1 Arithmetic Pipeline and Parameter Choices

The update equation we compute at each clock cycle is

$$u^{n+1} = \rho(u_L + u_R + u_U + u_D - 4u^n) + 2u^n - u^{n-1}(1 - 2^{-\text{ETA_SHIFT}}). \quad (9)$$

When we clamped all neighbors to zero, this simplifies to

$$u^{n+1} = \rho(-4u^n) + 2u^n - u^{n-1}(1 - 2^{-\text{ETA_SHIFT}}). \quad (10)$$

We parameterized the module with `rho = 18'sd8192`, which represents $\rho = 1/16$ in the 1.17 signed fixed-point format used throughout our design.

We portrayed damping using `ETA_SHIFT = 9`, approximating the physical damping coefficient as $\eta\Delta t/2 \approx 2^{-9} \approx 0.002$. The damping term $u^{n-1}(1 - 2^{-9})$ is computed with a single arithmetic right shift, which means it required no additional multiplier in our design. This was an explicit resource-budget decision on our part: because thirty such nodes would eventually run in parallel in the full drum, every saved DSP block means we are saving area on the FPGA. Our `signed_mult` module handles the ρ -scaling multiplication by multiplying two 18-bit 1.17 operands and producing a 36-bit product, from which bits [34:17] are extracted, maintaining the sign bit at position 35 and discarding the lowest 17 bits of fractional precision through truncation. This truncation introduces a small rounding error but avoids the additional logic required for proper rounding, a trade-off we accepted given that the 18-bit word width already provides sufficient resolution for audio-quality output.

The spatial Laplacian is calculated combinationally for each cycle. Instead of writing it as the conventional sum of neighboring differences, we equivalently express it as

$$\mathcal{L} = u_L + u_R + u_U + u_D - 4u_{\text{center}}, \quad (11)$$

where the factor of four is realized as a two-bit arithmetic left shift (`u_center <<< 2`), avoiding any additional multiplier. The entire combinational path from input registers to `u_center_next` therefore passes through exactly one DSP block and a handful of adder trees, giving a critical path that can be short enough to meet timing of 50 MHz with significant margin.

4.1.2 Initial Conditions and the Velocity Transient

We initialized the current and previous state registers to `18'sd16384`, encoding approximately 1/8 of full scale ($16384 \times 2^{-17} \approx 0.125$). The finite-difference time integration implicitly encodes instantaneous velocity as $(u^n - u^{n-1})/\Delta t$; initializing $u^n = u^{n-1}$ therefore corresponds exactly to a zero initial velocity condition, ensuring the membrane begins at rest at the specified displacement and rings down smoothly from the very first clock cycle. One of the errors we debugged by looking at our waveforms on ModelSim was that if we left `u_center_p` at zero while `u_center` was nonzero, the first update step would have interpreted the difference as a large abrupt velocity transient, injecting energy into the system at $t = 0$ with no physical justification and corrupting the early output samples in a way that would be difficult to distinguish from an arithmetic error during debugging.

4.2 Single Column Validation

Since we were able to verify our core update equation at a single node, we extended the design to a full column of $N = 30$ nodes. This step introduced two new types of complexity: on-chip memory for storing the two timesteps of state required by the second-order integrator, and a finite state machine which would sequence reads, computations, and writes across all the 30 rows in series. Instead of instantiating 30 copies of `wave_node` in parallel, which would consume 30 DSP blocks on the FPGA and a substantial amount of routing, we decided instead to use a single `compute_unit` combinationally, cycling it through each row in turn and storing each of the intermediate results in M10K block RAM. This serialization was significant to our strategy of reducing area: the full 30×30 grid would soon require 30 such columns running in parallel, so minimizing the per-column footprint was essential for fitting within the limits of the FPGA.

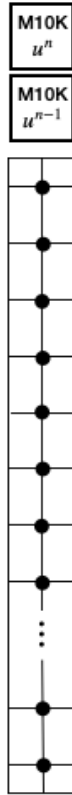


Figure 2: Column of $N = 30$ nodes implemented using a single compute unit and sequential memory access, adapted from [1].

4.2.1 Memory Organization

Each column stores two arrays of $N = 30$ values: the displacement at the current timestep u^n and the displacement at the previous timestep u^{n-1} . We implemented these as two dedicated M10K block RAM instances, `M10k_current` and `M10k_prev`, each parameterized as `M10K_540_18` with 18-bit-wide, 540-entry storage. Both blocks expose independent read and write address ports and register all outputs on the rising clock edge, which imposes

a one-cycle read latency that the FSM must explicitly account for. We chose to use two separate memories rather than a single dual-port RAM with interleaved addresses because we believed that this separation made the address sequencing logic much simpler to debug and reason about and it did not cost us area as both M10K blocks were necessary.

The synchronous update ordering that was enforced by the FSM is crucial for numerical accuracy. Each node’s updated value is written to `mem_curr` only after that node’s current value and all of the vertical neighbors have been read from memory. This makes sure that every node in a given timestep uses values from timestep n when computing timestep $n + 1$, and prevents the in-place update instability that would arise if a node’s freshly computed value were immediately visible to the node below it before that node had read its own input.

4.2.2 Compute Unit

The `compute_unit` module evaluates the full finite-difference update combinationally in a single cycle, accepting all five required values `u_above`, `u_below`, `u_left`, `u_right`, `u_center`, and `u_center_p`, and producing `u_center_next` with no internal registers. The discrete spatial Laplacian is computed as

$$\mathcal{L} = u_{\text{left}} + u_{\text{right}} + u_{\text{above}} + u_{\text{below}} - 4u_{\text{center}}, \quad (12)$$

implemented in Verilog as `u_left + u_right + u_above + u_below - (u_center <<< 2)`, where the factor of four is obtained by a two-bit left arithmetic shift. This result is scaled by ρ through the `signed_mult` DSP instance, and combined with the temporal terms to produce

$$u^{n+1} = \rho\mathcal{L} + 2u^n - u^{n-1}(1 - 2^{-\text{ETA_SHIFT}}), \quad (13)$$

where $2u^n$ is a one-bit left shift and the damping approximation is a right shift, keeping the total multiplier count at one per column. Because the `compute_unit` is purely combinational, it produces a valid output on every cycle as long as its inputs are stable, which the FSM guarantees by staging the correct memory outputs into the input registers one cycle before they are needed.

4.2.3 Control FSM

The `drum_col_control` module contains a six-state FSM that sequences through initialization, memory prefetching, and the row update loop (see Fig. 3). On reset, the FSM begins in the **INNIT** state, which writes a triangle amplitude profile into both `mem_curr` and `mem_prev` over 30 consecutive cycles. The triangle linearly increases from zero at row 0 to a peak at row $N/2 = 15$, then decreases back to zero at row $N - 1 = 29$, with the step size between adjacent entries set by the `step_size` parameter. Writing identical values to both memories sets $u^n = u^{n-1}$ for every node simultaneously, eliminating any initial velocity transient across the entire column.

Once initialization is completed, the FSM moves to **SETUP0**, which resets the row counter and issues the first read addresses to prime the M10K pipeline. Since the block RAM needs one full clock cycle before sending data at its output, the FSM must issue a read address one cycle before that data is needed by the `compute_unit`. **SETUP1** absorbs this

latency by issuing a second prefetch before any computation begins, ensuring that valid data is present at the memory outputs by the time the main loop begins. **PIPELINE_PRIME** stages the first row's neighbor values from memory into the FSM's output registers, handling the bottom boundary by substituting 18'sd0 for `u_below` at row 0.

The main update loop runs in **SWEEP** for $N - 1 = 29$ cycles. On each cycle, the FSM advances the memory read addresses, stages `u_above`, `u_center`, and `u_below` from the pipeline, passes them to the `compute_unit`, and writes the resulting `u_center_next` back to `mem_curr` while simultaneously archiving the outgoing `u_center` to `mem_prev`. Boundary conditions are enforced by substituting 18'sd0 for `u_above` when the counter reaches $N - 2$, clamping the top edge of the column to zero displacement. The center node amplitude is tapped when `row_counter == N/2` and held in `test_out` for output. Finally, **FLUSH** commits the last pipelined write that SWEEP could not complete within its loop bounds, then returns the FSM to **WAIT** to begin the next timestep right away.

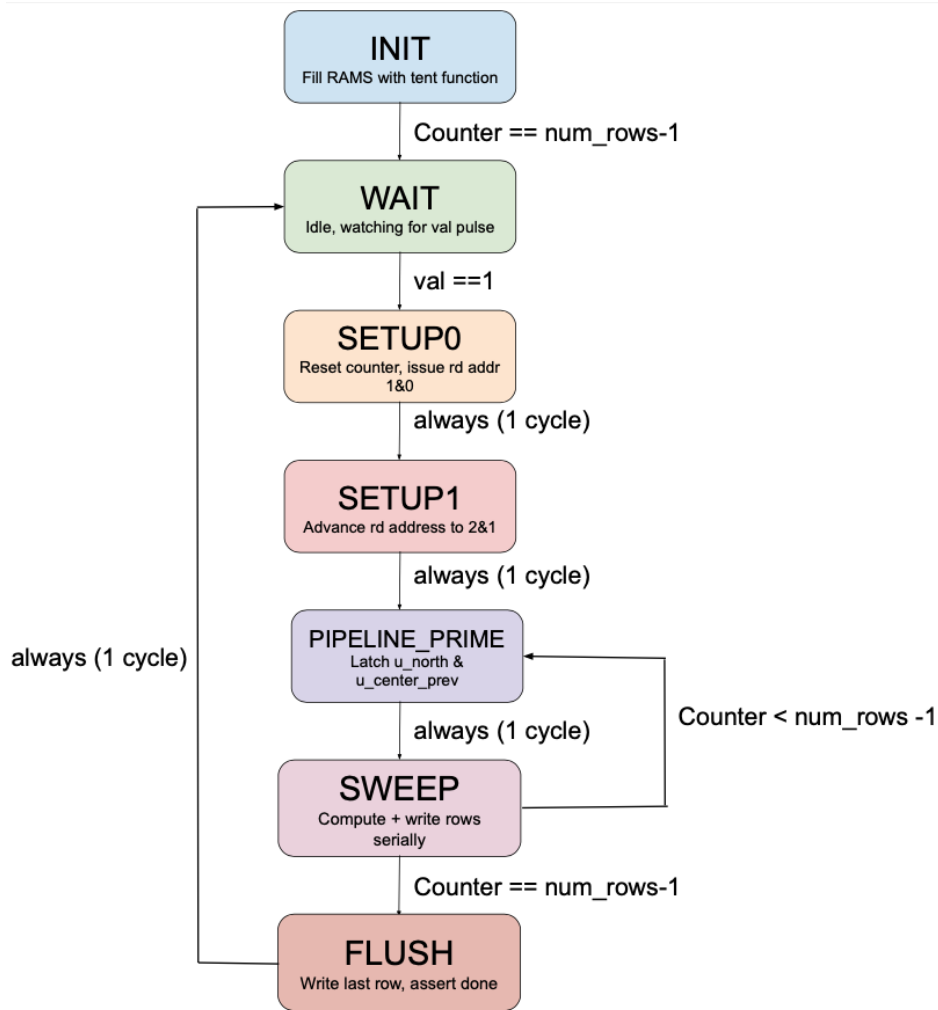


Figure 3: Finite state machine for the `drum_col_control` module.

4.3 Full 30×30 Drum Validation

4.3.1 Parallel Column Instantiation and Lateral Coupling

We instantiated all 30 columns using a Verilog `generate` block parameterized by `NUM_COLS = 30`. Every column runs its own FSM independently and all the FSMs are driven by the same clock and `val` strobe, so they advance in perfect lockstep: when column c is processing row r , every other column is also processing row r . This synchrony is what makes lateral coupling straightforward and comprehensible, so at the moment column c needs `u_left` and `u_right` for its current row, columns $c-1$ and $c+1$ are guaranteed to have already staged the correct row value into their `u_curr` output register, since all columns began their row sweep at the same time. The `u_curr` signal is driven from `u_center`, the registered center-node value held in the FSM output registers, so the lateral neighbor presented to adjacent columns always represents the value that the `compute_unit` is currently operating on for that row.

We wired lateral coupling by connecting each column's `u_curr` output directly to the `u_left` input of the column to its right and the `u_right` input of the column to its left. At the left and right boundaries ($c = 0$ and $c = 29$), we tied the lateral inputs to `18'sd0`, and setting fixed Dirichlet boundary conditions on all four edges of the grid:

```
.u_left (c == 0 ?  
        18'sd0 : u_curr_bus[c-1 < 0 ? 0 : c-1]),  
.u_right(c == NUM_COLS - 1 ?  
        18'sd0 : u_curr_bus[c+1 > NUM_COLS-1 ? NUM_COLS-1 : c+1]),
```

Because the neighbor values are read combinationally from adjacent columns' registered outputs, lateral coupling adds no additional pipeline stages and introduces no serialization penalty. The full 30×30 grid therefore advances one complete timestep in exactly the same number of clock cycles it takes a single column to sweep its 30 rows, giving us a throughput of one timestep per column sweep no matter the number of columns.

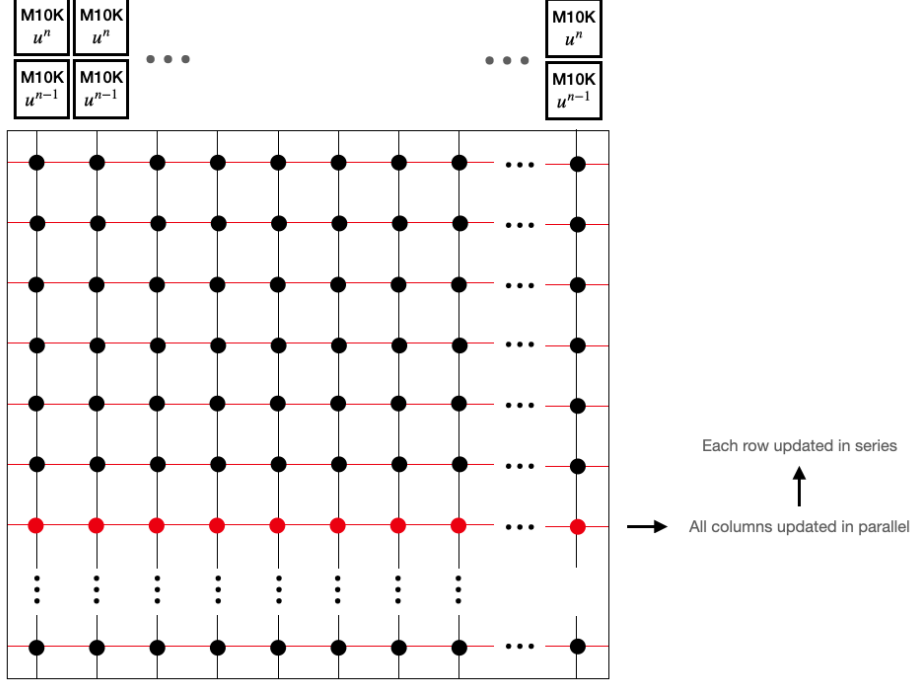


Figure 4: 30×30 drum architecture with columns operating in parallel and rows processed sequentially. Lateral coupling is achieved through direct neighbor connections between adjacent columns, adapted from [1].

4.3.2 Updated Compute Unit and Damping

When we scaled up to the full grid we were prompted to make some adjustments to our `compute_unit`. In the single-node and single-column validations, damping was applied only to the previous-timestep term u^{n-1} , using `ETA_SHIFT = 9`. In the full drum, we apply a second damping shift directly to the output u^{n+1} as well, which gives us the update equation

$$u^{n+1} = \left[\rho \mathcal{L} + 2u^n - u^{n-1}(1 - 2^{-13}) \right] (1 - 2^{-13}), \quad (14)$$

implemented in Verilog as

```
assign center_prev_damped = u_center_p - (u_center_p >>> 13);
assign u_next_raw         = u_rho_two_center - center_prev_damped;
assign u_center_next      = u_next_raw - (u_next_raw >>> 13);
```

The shift constant was changed from 9 to 13 (approximating $\eta \Delta t / 2 \approx 2^{-13} \approx 0.00012$) to reduce the damping rate and allow the better and richer multimodal oscillation of the 2D membrane to ring out over a longer, more useful window. Applying the shift twice in sequence, once to the previous-state term and once to the final output, introduced a small additional energy loss per step, which in practice acts as a mild low-pass stabilizer across the grid without altering the modal frequencies.

4.3.3 FSM Changes: WAIT State and val Handshake

The single-column FSM used a direct loop from **FLUSH** back to **SETUP0**, advancing immediately into the next timestep without any external synchronization. For the full drum we inserted a new **WAIT** state between **INNIT** and **SETUP0**, and also as the landing state after **FLUSH** completes. In **WAIT**, the FSM holds all write-enables that were deasserted and does not move on until the external **val** strobe is asserted for one cycle. This handshake makes sure we decouple the column’s internal timing from whatever upstream consumer (the audio DAC interface or a testbench) is reading the **test_out** samples. A column signals completion of its current timestep by asserting **done** in **FLUSH**; the top-level logic can then confirm all 30 **done** signals before issuing the next **val** pulse, guaranteeing that no column begins a new timestep until every column has finished the current one. The FSM now cycles through seven states in total: **INNIT** → **WAIT** → **SETUP0** → **SETUP1** → **PIPELINE_PRIME** → **SWEEP** → **FLUSH** → **WAIT**, with **SWEEP** iterating for $N - 1 = 29$ cycles before transitioning.

4.3.4 Pyramid Initial Conditions

Now we had to extend the per-column triangle initialization from Section 4.1(drum column) to two dimensions, we constructed a pyramid initial condition: the product of a row-wise triangle envelope and a column-wise triangle envelope, which produced a displacement surface that peaks at the center of the grid and tapers smoothly to zero on all four edges. We made this choice based on two motivations. First, a smooth initial condition excites only the low-order normal modes of the 2D grid, avoiding the high-frequency spatial aliasing that a point excitation at the center node would bring. Second, the pyramid naturally satisfies the fixed boundary conditions on all four edges without any special-casing, since both triangle envelopes evaluate to zero at the boundaries.

We implemented the column-wise envelope by varying the **step_size** parameter passed to each column’s FSM according to

$$\text{step_size}[c] = \begin{cases} \text{step_size}[c-1] + \text{base_step} & c \leq \text{NUM_COLS}/2 \\ \text{step_size}[c-1] - \text{base_step} & c > \text{NUM_COLS}/2, \end{cases} \quad (15)$$

computed in the **drum_top** generate block as

```
assign step_size[c] = (c <= (NUM_COLS >> 1))
    ? step_size[c-1] + base_step
    : step_size[c-1] - base_step;
```

so that column 15 at the center receives the largest step size of $512 \times 15 = 7680$, producing a peak row amplitude of $15 \times 7680 = 115,200$ (approximately 0.88 in normalized 1.17 scale), while the boundary columns at $c = 0$ and $c = 29$ receive a step size of zero. The row-wise triangle in each column is controlled by the same FSM initialization loop described in Section 4.1, linearly ramping up from row 0 to row 15 and back down to row 29. The result is a smooth bilinear pyramid across the full grid surface that stays well within the representable range of our 18-bit fixed-point format at all nodes.

4.3.5 Center Node Output and Observed Behavior

We routed the amplitude signal to the audio DAC from `test_out_bus[NUM.COLS >> 1]` in `drum_top`, corresponding to the node at column 15, row 15, the geometric center of the grid. This tap is captured inside SWEEP of each column’s FSM when `row_counter == N/2`, meaning it reflects the value of `u_center_next` at the midpoint of the row sweep, which coincides exactly with the center node for a 30×30 grid.

In ModelSim, the center node output exhibited rich damped multimodal oscillation rather than the single-frequency sinusoid we observed in the isolated column. This was expected: the pyramid initialization simultaneously excites several low-order normal modes of the 2D membrane, each with its own frequency and decay rate, and their superposition produces a more complex waveform whose envelope decays over time. The absence of high-frequency noise or sharp discontinuities in the output confirmed that the smooth initial condition successfully suppressed the aliased modes that a point excitation would have introduced. We also verified in simulation that all 30 `done` signals asserted simultaneously each timestep, confirming that the lockstep synchronization between columns was maintained throughout the run and that no column fell behind or ahead of its neighbors.

5 Parallelization Scheme

One of the key objectives of our design was to maximize the size of the simulated drum while Making sure we had real-time performance at a 48 kHz audio rate. Therefore we developed a hybrid parallelization strategy that exploited spatial parallelism across columns while Maintaining and preserving resource efficiency through temporal serialization within each column.

5.1 Column-Level Parallelism

In our design of the full 2D drum, we decomposed it into a set of independent vertical columns, each of which is implemented as a separate `drum_column` module. We instantiated these columns using a Verilog `generate` block, which allowed us to scale the system to a large number of columns (up to 128 in our final design) while having identical structure across all of the instances.

Each of the drum columns operated in parallel with all the other columns and each contained its own:

- `compute_unit` (finite-difference solver),
- `drum_column_control` FSM,
- two M10K memory blocks storing u^n and u^{n-1} .

Since each column had a dedicated compute unit and memory, all the columns could process the same row index simultaneously. This allowed our system to compute one full timestep across all columns in parallel, which significantly increased throughput compared to a fully serialized approach.

5.2 Row Serialization Within Columns

While the columns operated in parallel, each of the columns processed its rows sequentially using a single combinational `compute_unit`. The FSM in each column iterated through all N rows, and performed one node update per clock cycle once the pipeline was filled.

We chose this serialization strategy in order to minimize hardware resource usage. Instantiating a fully parallel $N \times N$ grid of compute units would require one DSP block per node, which is unreasonable for large grids. By reusing a single compute unit per column, we reduce the DSP usage to one per column while still having high throughput through column-level parallelism.

As a result, the time that was required in order to compute one timestep was proportional only to the number of rows (N), and was independent in relation to the number of columns.

5.3 Synchronous Column Operation and Data Coupling

All columns are driven by a shared clock and advance in lockstep under control of the `val` handshake signal. This synchronization allows that when a column is processing a given row, its neighboring columns are processing the same row at the same time.

This lockstep behavior allows for efficient lateral coupling between the drum columns. The center-node value (`u_curr`) from each drum column is exposed as an output and directly connected to the neighboring drum columns as `u_left` and `u_right`. Since these values are registered outputs, they are stable and valid when required, and allowed neighbor interactions to be implemented without additional buffering or pipeline stages needed.

We enforced the boundary conditions by setting the outermost column inputs to zero, ensuring fixed-edge behavior across the grid.

5.4 Throughput and Scalability

The combination of column-level parallelism and row-level serialization helped us produce a highly scalable architecture. For a grid of size $C \times N$, the system computes one full timestep in $O(N)$ cycles, regardless of the value C . This allowed our design to scale to a large number of columns without needing to increase computation time per timestep.

This design efficiently utilizes DSP blocks, M10K memory, and logic elements, and accentuates the effectiveness of structured parallelization for solving partial differential equations on FPGA hardware.

5.5 Scaling

Once we had our 30x30 drum working, the last thing to do was make it larger. The scaling process was relatively simple because we had already parameterized our design on a number of columns and a number of rows.

First, we wanted to maximize the resource usage on the FPGA. We scaled up incrementally - trying 64, 100, then 128 columns. We also tried 150 columns, but discovered that we were memory constrained. Due to long compilation times, we decided to stick with 128 columns rather than spend hours compiling for an additional few columns.

After we settled on 128 columns, we scaled the number of rows. Because the number of rows only determines how many times our FSM runs in a time step, we realized that we could change it without recompiling. To do this, we tied the `num_rows` input to our top level drum module to the first 8 switches on the DE1 board. Once we compiled this design, we could increase rows one by one, until we found the breaking point. Interestingly, we found a hard break at 101 rows. 100 rows sounds great (aside from the occasional terrible screeching), but 101 rows no longer makes any sound at all. We attempted to further increase the rows by adding a second PLL at a higher clock rate, but we still were not able to get past 100 rows. We later discovered that our design could do 100 rows in just over 2 microseconds. We found it curious that this hard break occurred at such a seemingly 'random' number, despite our synth meeting timing at 100 rows easily (see Section 7).

6 Design and Testing Methods

6.1 Hardware Testing

Before we had compiled our code and pushed it to the FPGA, we conducted extensive functional validation using ModelSim. These simulations emulated the behavior of the hardware at the RTL level and allow us to see internal signals such as node displacements, neighbor interactions, and memory updates over time. We performed the testing process in a bottom-up way, beginning with a single node, then extending to a full column, and finally to the complete two-dimensional drum system.

6.1.1 Single Wave Node Verification

We first validated the correctness of the discrete wave equation of a single `wave_node`. The testbench initialized the node with a nonzero displacement while setting neighboring values appropriately to observe the resulting oscillatory behavior. The output waveform demonstrated that the node evolves over time according to the expected second-order update equation, exhibiting stable oscillations and amplitude that was bounded.

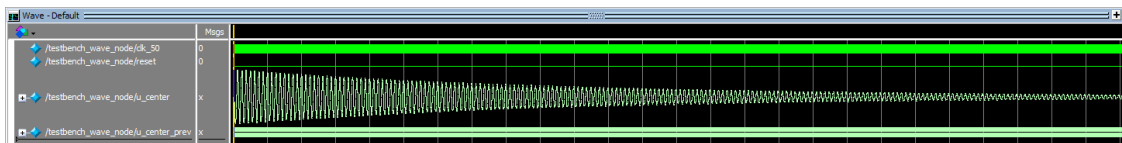


Figure 5: ModelSim waveform showing the temporal evolution of a single wave node.

6.1.2 Drum Column Verification

After we had confirmed that we were getting expected behavior for a single node, we moved on to testing the full column design implemented by the `drum_col_control` module. The simulation verified that the FSM correctly sequenced through initializations, memory prefetchings, and the sweep phase while keeping the data flow between `mem_curr`, `mem_prev`, and the `compute_unit` correct and accurate.

The waveform on ModelSim confirmed that values propagated along the column as expected, with boundary conditions enforced at the top and the bottom nodes. Furthermore, the center node output (`test_out`) exhibits oscillatory behavior consistent with wave propagation in a discretized medium.

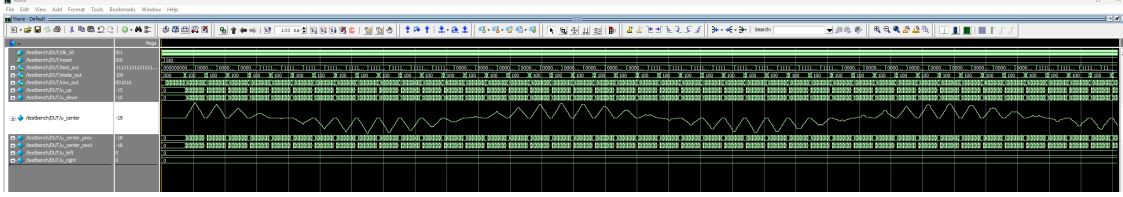


Figure 6: ModelSim waveform for the drum column at initialization, showing the initial displacement profile and memory preloading behavior.

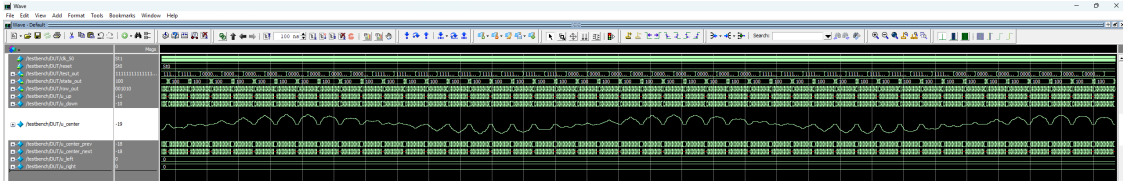


Figure 7: ModelSim waveform for the drum column after several nanoseconds, illustrating wave propagation and intermediate node updates across the column.

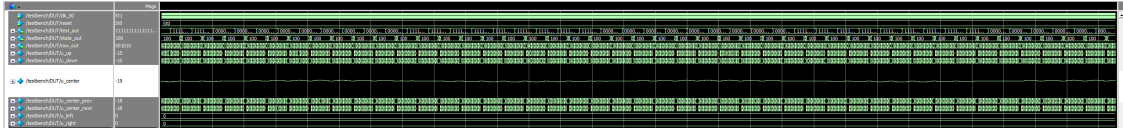


Figure 8: ModelSim waveform for the drum column at a later time, demonstrating sustained oscillatory behavior and stability of the system.

6.1.3 2D Drum System Verification

Finally, we tested the complete 30×30 drum system composed of multiple columns operating in parallel. The simulation showed that wave propagation occurs across both spatial dimensions, producing the expected drum-like vibration patterns. The interaction between neighboring columns confirmed that there was correct coupling behavior, while the overall system remained stable over time.

The waveform illustrates the evolution of selected nodes across the grid, showing coherent oscillatory motion and proper boundary enforcement at the edges of the drum.

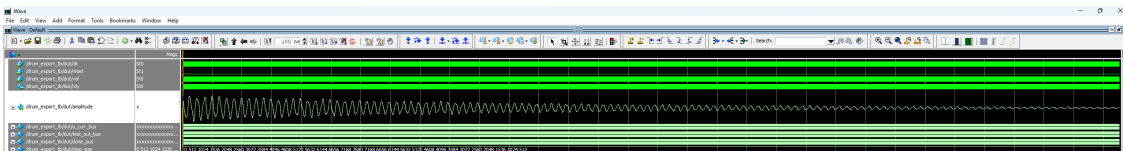


Figure 9: ModelSim waveform of the full 2D drum system showing spatial wave propagation.

6.2 Audio Output Validation

To further validate the physical realism of the 2D drum simulation, we extracted the time-varying amplitude of the center node from the full 30×30 grid during ModelSim simulation. These amplitude values were recorded over many timesteps and we exported the ModelSim data as a discrete time-series dataset. A Python script was then used to post-process this data by scaling and converting these fixed-point values we had from the simulation into a standard audio format. We played the resulting waveform through speakers, allowing us to directly hear the simulated vibration of the drum.

The sound we heard was a sharp, metallic timbre resembling a cowbell or woodblock, which seemed consistent with the expected acoustic behavior of a discretized membrane with fixed boundary conditions and little damping.

Once we had our design running on the DE1, we compared the synthesized audio to the audio generated by our simulation and decided they were similar. Additionally, we used an oscilloscope to record the actual waveform being sent to the Audio Codec in realtime (Figure 11).

6.3 Software Testing

The only portion of this lab that we used the HPS for was running the 30x30 drum C program. To test that this program was able to compute the full drum at audio synthesis rate (48KHz), we probed the main synthesis loop to measure execution time using the linux realtime clock. Listing 1 demonstrates the modifications we made to test that the HPS was able to compute a 30x30 mesh in under $20.8 \mu s$. Our results showed that the HPS finished computation consistently around 20-21 μs , and the output sounded correct. Given the latency introduced by the print statement and the timing overhead, we are confident that the HPS code is meeting timing. We also used this data to compare the speed of the HPS drum to our accelerated design - discussed in Section 10.1.

```
1 while (((*audio_fifo_data_ptr>>24) & 0xff) > 1) {
2
3     // === START timing one drum time step ===
4     clock_gettime(CLOCK_MONOTONIC, &ts_start);
5
6     for (i=1; i<drum_size-1; i++){
7         for (j=1; j<drum_size-1; j++){
8             new_drum_temp = times_rho(drum_n[i-1][j] + drum_n[i+1][j]
9                                     + drum_n[i][j-1] + drum_n[i][j+1] - times4pt0(
10                                     drum_n[i][j]));
11             new_drum[i][j] = times0pt9999(new_drum_temp + times2pt0(
12                                     drum_n[i][j]) - times0pt9998(drum_n_1[i][j]));
13         }
14     }
15     memcpy((void*)drum_n_1, (void*)drum_n, copy_size);
16     memcpy((void*)drum_n, (void*)new_drum, copy_size);
17
18     clock_gettime(CLOCK_MONOTONIC, &ts_end);
```

```

16 // === END timing ===
17
18 elapsed_ns = (ts_end.tv_sec - ts_start.tv_sec) * 1000000000L
19             + (ts_end.tv_nsec - ts_start.tv_nsec);
20 printf("One drum step: %ldns (%.3fus)\n", elapsed_ns,
21        elapsed_ns / 1000.0);
22
23 *audio_left_data_ptr = fix2audio16(drum_n[drum_middle][
24    drum_middle]);
25 *audio_right_data_ptr = fix2audio16(drum_n[drum_middle][
26    drum_middle]);
27 }

```

Listing 1: 30x30 HPS Code - modified to probe timing

7 Results

The largest drum grid we were able to build was 128 columns by 100 rows. The number of columns we could achieve was limited by the resources present on the FPGA - notably M10K memory and area. When trying to compile a design with 150+ columns, the compiler failed during synthesis because it tried to use more than the available 3970Kb of M10K memory available on the DE1-SoC. We were also pushing close to 80% ALM utilization with 128 columns, meaning we didn't have much headroom in that department either.

We anticipated that our number of rows would be limited by timing. However, we believe that there is some other flaw in our design that kept us capped around 100 rows (beyond which the drum started to screech). As seen in Figure 10, the 100 row drum finished synthesis in $2.07\mu s$, well under the $20.08\mu s$ period of the 48KHz audio synthesis rate. This indicates that our state machine would be capable of calculating nearly 1000 rows (10 times 100) within the given time constraint. In reality, the size of each M10K block would need to scale with the number of rows - so we would have likely run out of memory before this point. Regardless, our design only being capable of 100 rows indicates another bug in our system (potentially a bit width mismatch, or something else) that we unfortunately did not have time to locate and fix.

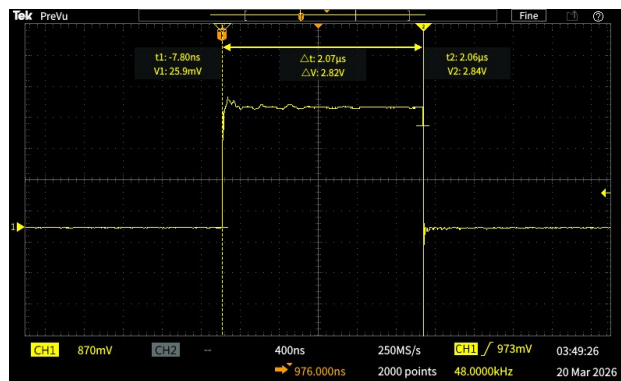


Figure 10: Oscilloscope trace showing runtime of a 100 row drum.

The audio produced by our 128x100 drum has the waveform shown in Figure 11. The sound was deep and reverberating - it most closely resembled the sound of a fairly large timpani.

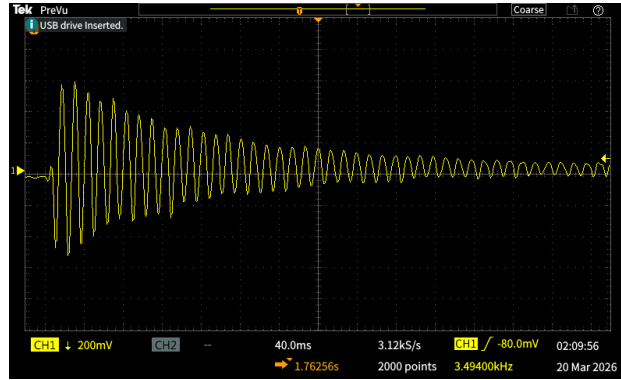


Figure 11: Oscilloscope trace waveform of 128x100 drum.

8 Chip Planner Layout

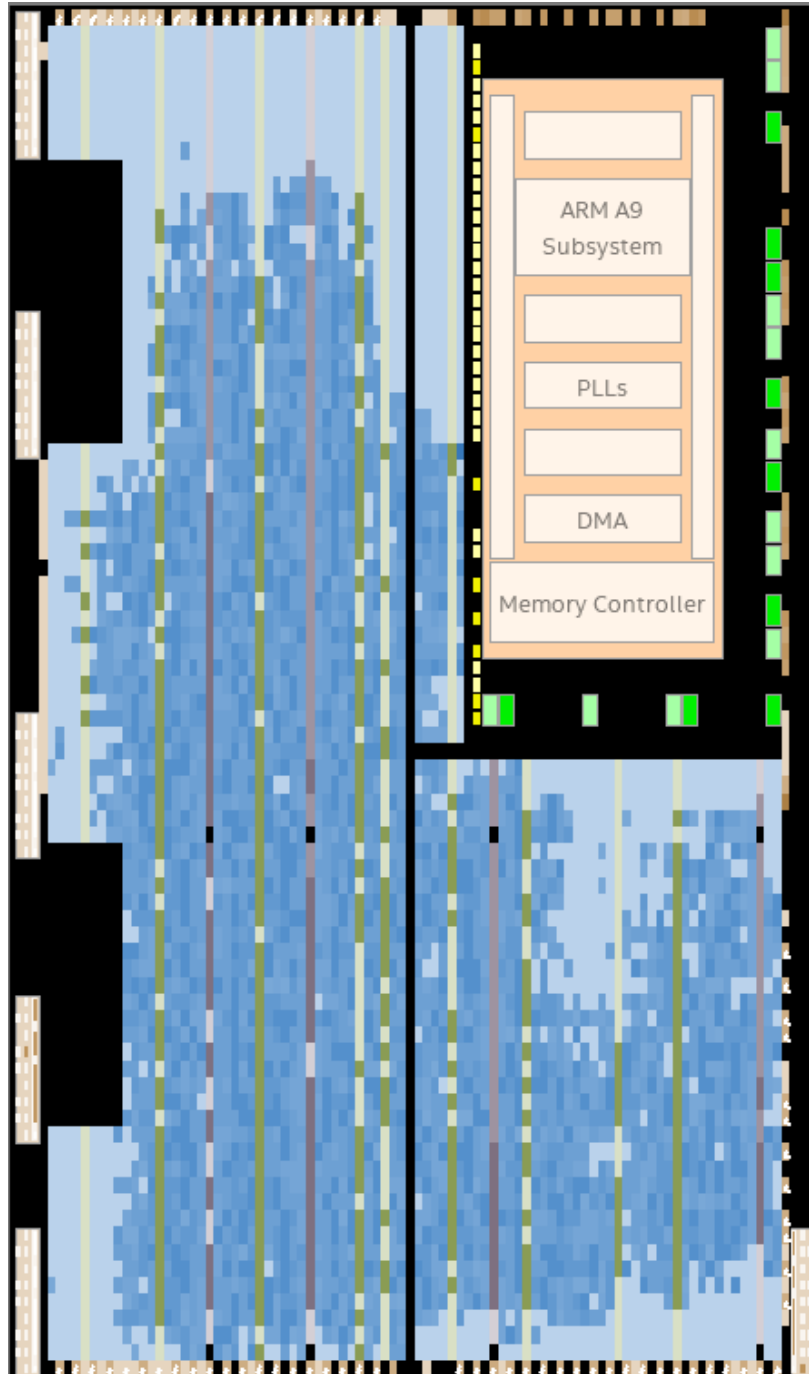


Figure 12: FPGA Chip Planner showing resource utilization of a 128 column drum.

Figure 12 shows the Chip Planner view generated by Quartus, providing a physical representation of how the drum synthesis accelerator is mapped onto the FPGA fabric of the DE1-SoC. Unlike the RTL schematic, which illustrates the logical connectivity of the system, the Chip Planner reveals the physical placement of logic elements, embedded memory

blocks, and routing resources after synthesis, placement, and routing have been completed.

The large rectangular region on the right side of the planner corresponds to the Hard Processor System (HPS), which contains the ARM A9 subsystem, PLLs, and memory controller. This region is fixed silicon and is not part of the programmable FPGA fabric. The remaining area represents the programmable logic where our drum synthesizer was implemented.

The blue regions scattered across the FPGA indicate utilization of adaptive logic modules (ALMs) and registers. In our drum synthesis design, these resources correspond to the per-column `drum_column_control` state machines, the Avalon bus master driving the audio FIFO, and associated control logic. Because `drum_top` instantiates 128 parallel `drum_column` modules - each containing its own `compute_unit` and FSM - the ALM usage is replicated uniformly across the fabric, which is reflected in the distributed blue regions visible throughout the layout.

The green vertical columns represent M10K embedded memory blocks. Each `drum_column_control` instance maintains two M10K memories — one storing the current displacement values (`mem_curr`) and one storing the previous time-step values (`mem_prev`) — enabling the second-order finite difference time-domain update at every sample. With 128 columns instantiated, the M10K blocks constitute a significant share of the on-chip memory budget and are visible as dense vertical structures distributed across the chip.

From the layout, the synthesis and placement tools distributed logic and memory resources broadly across the FPGA fabric rather than concentrating them in a single region. This distribution is a natural consequence of the highly parallel column architecture: each column's compute logic is placed in proximity to its associated M10K pair, reducing the routing distance between the FSM, compute unit, and memory. The routing paths visible throughout the device illustrate signal flow between adjacent columns, which is necessary for the lateral coupling terms (`u_left`, `u_right`) in the 2D wave equation update.

Poor placement of the column instances could introduce long routing paths between laterally coupled neighbors, increasing propagation delays and risking timing violations at 50 MHz. As shown in the Chip Planner, however, the design successfully maps to the FPGA fabric with logic and memory resources efficiently distributed, supporting the real-time parallel computation required to produce continuous audio output through the Avalon audio interface.

9 Hardware Resource Utilization

Above in Figure 13 is the compilation report produced by Quartus for our 128 column drum. For this design, we used 76% (or 24,308) of the available ALMs on the DE1-SoC. This indicates that we still had some room to grow with this design, however routing would become increasingly difficult as the remaining area shrinks. We also were memory constrained past 128 columns - so we believed our main constraint was M10K memory rather than logic elements.

Flow Status	Successful - Wed Mar 25 14:32:15 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	DE1_SoC_Computer
Top-level Entity Name	DE1_SoC_Computer
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	24,308 / 32,070 (76 %)
Total registers	24514
Total pins	368 / 457 (81 %)
Total virtual pins	0
Total block memory bits	942,592 / 4,065,280 (23 %)
Total DSP Blocks	87 / 87 (100 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	2 / 6 (33 %)
Total DLLs	1 / 4 (25 %)

Figure 13: Hardware Utilization

9.1 Program Listings

Module	Description
<code>signed_mult</code>	Single-cycle Q1.17 fixed-point multiplier. Takes two 18-bit signed operands and returns the correctly shifted 18-bit product, used throughout the wave-equation computation.
<code>M10K_540_18</code>	Dual-port M10K on-chip memory wrapper (18-bit wide, 201 entries). Provides separate read and write addresses with synchronous reads, instantiated twice per column to store the current and previous displacement arrays. Size adjusted from default 540 rows to reduce memory constraint.
<code>compute_unit</code>	Combinational 2D wave-equation finite-difference solver. Given the five nearest-neighbour values and the previous centre value, computes the next displacement using a configurable <code>rho</code> gain and a light damping term.
<code>drum_column_control</code>	FSM that serially sweeps through all rows of one drum column each cycle. Handles memory initialisation with a triangular-pyramid initial condition, then on each <code>val</code> pulse reads neighbours, drives <code>compute_unit</code> inputs, and writes results back to both M10K banks.
<code>drum_column</code>	Structural wrapper combining one <code>compute_unit</code> and one <code>drum_column_control</code> into a self-contained column. Exposes the centre-node value (<code>u_curr</code>) for lateral coupling to adjacent columns.
<code>drum_top</code>	Parameterised top-level drum model. Instantiates <code>NUM_COLS</code> (default 128) <code>drum_column</code> instances with a generated pyramid step-size initial condition and wires lateral boundaries to zero. Exposes the centre-column amplitude as the audio output sample and <code>done_bus[0]</code> as the ready strobe.

Table 1: HDL Module listing and definition.

9.2 User Interface

The user interface for our drum synth is very simple and relies entirely on physical inputs from the DE1-SoC board. We do not do any control of the drum synth from the HPS. Figure 14 demonstrates each component on the DE1 board that we use, labeled with their corresponding purpose. Table 2 details the purpose of each part of our interface.

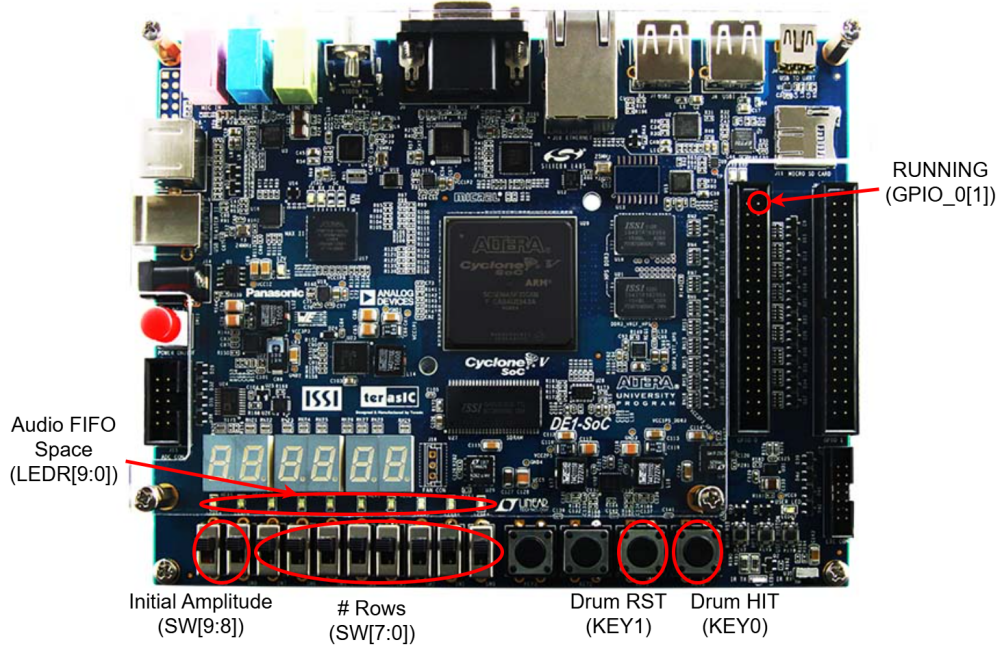


Figure 14: DE1-SoC overview labeled with controls for the Drum Synth.

Name	Component	Usage
Audio FIFO Space	LEDR[7:0]	Displays remaining space in the audio FIFO, for debugging.
Initial Amplitude	SW[9:8]	Pick initial amplitude for drum strike from 4 preset values.
# Rows	SW[7:0]	Pick number of rows to compute.
Drum RST	KEY1	Reset drum to zero amplitude.
Drum HIT	KEY0	Hit drum (set amplitude to preset, and run).
RUNNING	GPIO_0[1]	GPIO set when synthesis for one timestep is running - for timing.

Table 2: Listing of all components used for our UI and descriptions.

10 Documentation and Answers to Lab Questions

10.1 Calculation Times

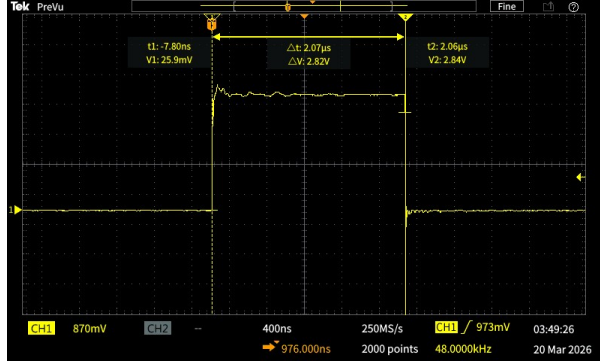


Figure 15: Scope trace of the 'running' signal in our drum FSM. For a 30x100 drum.

Row Count	Runtime
30 (HPS)	20.970 μ s
2	113ns
30	657ns
100	2.07 μ s

Table 3: Calculation time for various row counts.

To determine the runtime of our drum, we added an additional register to our FSM that is set high in the starting state, and cleared in the done state. A trace of this result is shown in Figure 15. We did this calculation for three different row sizes (all with 30 columns). The results of this experiment can be seen in Table 3.

From this table, it is obvious that we have plenty of headroom remaining (time-wise) to fit into the 48KHz audio synthesis rate used in this lab.

Additionally, compared to the 30x30 drum synthesized on the HPS we achieved a 31.9 X speedup for the same size drum. This comparison emphasizes the capabilities of hardware acceleration and parallelization.

10.2 Frequency Analysis

Using the FFT function on the TBS1052C Oscilloscope, we were able to quickly find the power spectrum for a range of drums. For various drum sizes, we have included an FFT decomposition of the output in Figures 16 to 19. We see the dominant (fundamental) frequency decrease as drum size increases. Table 4 shows the value of the apparent fundamental frequency in each of these plots.

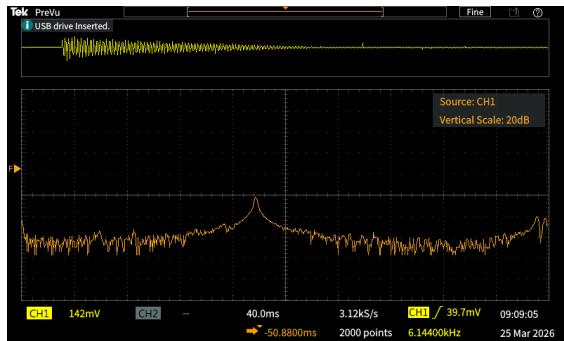


Figure 16: Power spectrum, 30x30.

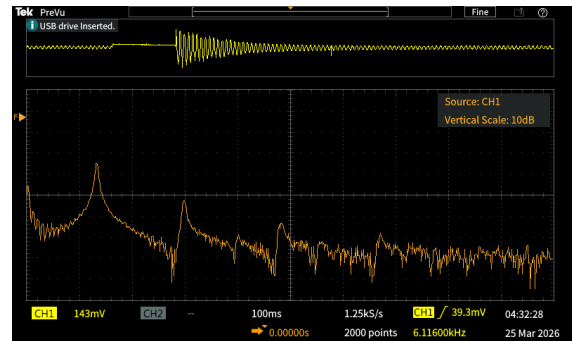


Figure 17: Power spectrum, 100x100.

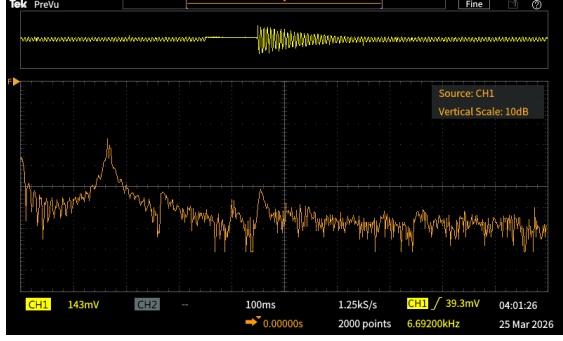


Figure 18: Power spectrum, 128x64.

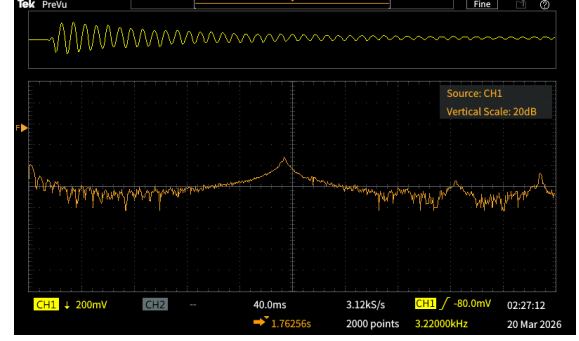


Figure 19: Power spectrum, 128x100.

Drum Size	Apparent Fundamental
30 x 30	276 Hz
100 x 100	82.5 Hz
128 x 64	103 Hz
128 x 100	75.9 Hz

Table 4: Fundamental frequencies for 4 drum sizes.

10.2.1 Apparent vs Theoretical Drum Modes

The largest square drum we could run was 100 by 100. For this size drum, we calculated the theoretical fundamental frequency using the equation for a square membrane. Since our FPGA drum synthesizer implements $\rho = \frac{1}{16}$, the effective wave speed in the finite-difference scheme is:

$$c_{eff} = \sqrt{\rho} \cdot \frac{F_s}{n} = \sqrt{\frac{1}{16}} \cdot \frac{48000}{100} = 120 \text{ grid units/s}$$

Substituting into the fundamental frequency equation for a square membrane:

$$f_0 = \frac{\sqrt{2}}{2L} \cdot c_{eff} \approx 84.9 \text{ Hz}$$

Using this corrected theoretical fundamental frequency, we used the Python script provided in the lab handout [1] to plot the power spectrum and mark the location of the expected harmonics (Figure 20). Using the FFT function on the oscilloscope, we were able to derive the apparent fundamental frequency, along with a few of the obvious harmonics. A comparison of the numerical values can be found in Table 5.

The measured fundamental agrees closely with the theoretical value. For the higher modes, not all theoretical harmonics were distinguishable on the oscilloscope due to noise. By matching each apparent peak to its closest theoretical harmonic, we identify that the first four visible harmonics closely correspond to the 3rd, 5th, 8th, and 13th theoretical modes, with the intermediate modes present but not resolvable in the spectrum. This matching is shown in Table 5. The Mode Ratios used in this table were taken from S. A. Van Duyne [2]

Based on discussions with Hunter and Bruce, we believe the pyramid initial condition in our design preferentially excites certain harmonics over others. The sharp discontinuities at the pyramid’s corners likely suppress some vibrational modes while emphasizing others. This contrasts with the Gaussian initial condition used in the theoretical Python model, which has no discontinuities and therefore excites all harmonics more uniformly. While this remains a hypothesis, it could be tested by experimenting with different initial conditions in the drum synthesizer — something we did not have time to explore.

Mode	Mode Ratio	Theoretical (Hz)	Apparent (Hz)	Error (%)
Fundamental	1.000	84.9	82.5	2.8%
3rd harmonic	2.245	190.2	186	2.2%
5th harmonic	2.920	247.9	251	1.2%
8th harmonic	3.540	300.5	302	0.5%
13th harmonic	4.120	349.8	351	0.3%

Table 5: Apparent oscilloscope measurements matched to their closest theoretical drum modes for a 100x100 drum with $\rho = \frac{1}{16}$.

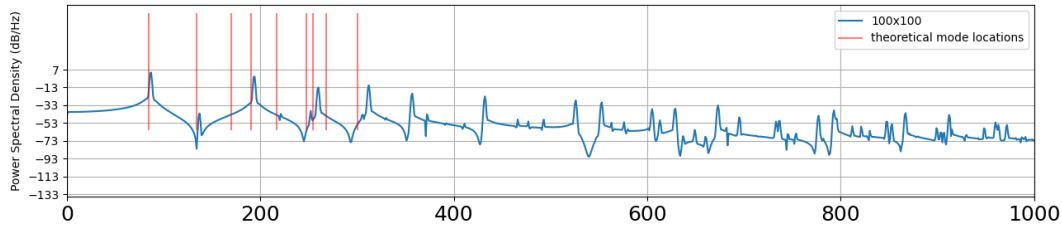


Figure 20: Theoretical modes of a 100x100 drum with $\rho = \frac{1}{16}$, plotted.

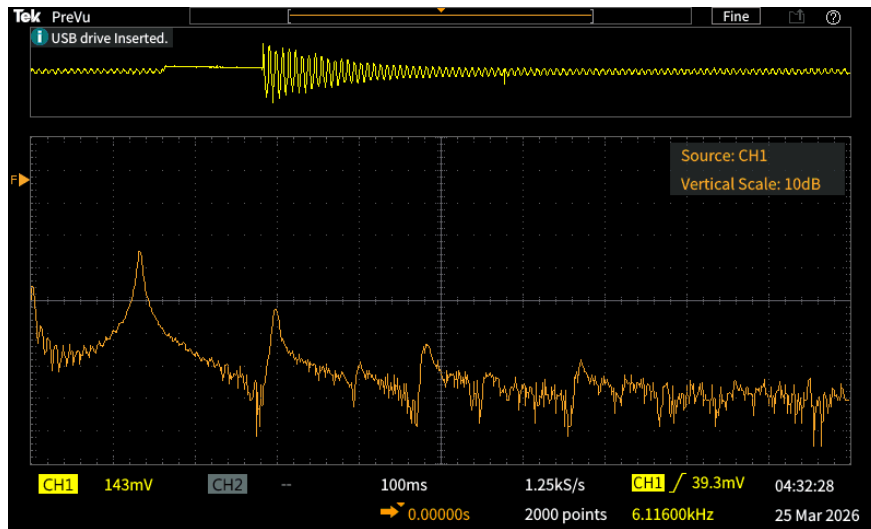


Figure 21: Power spectrum of our 100x100 drum.

10.3 Platform Designer

For this lab, our Avalon Bus is relatively simple. The only devices of interest on the bus are the Audio_Subsystem, the AV_Config, and the EBAB Bus_master_audio. The EBAB is the bus master through which we can interact with the Audio Core Subsystem and by extension the audio codec onboard the DE1-SoC. This bus master exposes 6 signals to our top level design that we use to manipulate the audio slave. These signals are enumerated in table 6.

Our state machine design interacts with the audio subsystem by using its external bus-master interface to read the audio FIFO status and write new sample data when space is available. In each output cycle, the state machine first reads the FIFO-space register to determine whether the audio core can accept another sample. If enough room exists, the design generates a new audio sample and writes it to the left and right channel data registers, which place the sample into the audio core's output FIFOs. The audio subsystem then removes samples from these FIFOs at the codec's playback rate and transmits them to the DE1-SoC audio codec for analog output.

Signal	Description
bus_addr	Address of the Avalon-MM register being accessed.
bus_byte_enable	Selects which bytes of the 32-bit word are active in the current transfer.
bus_read	Asserted to request a read from the addressed register.
bus_write	Asserted to request a write to the addressed register.
bus_write_data	32-bit data placed on the bus during a write transaction.
bus_ack	Handshake signal indicating the current bus transaction has completed.
bus_read_data	32-bit data returned by the bus during a completed read transaction.

Table 6: Simplified EBAB audio bus signals.

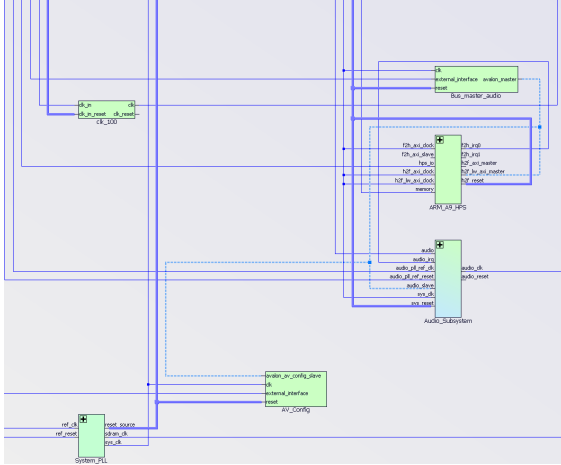


Figure 22: Quartus block diagram of Avalon Bus.

10.4 RTL View

Here we've included a few screenshots from the RTL Viewer to show the footprint of our drum state machine and (30 column) top level design.

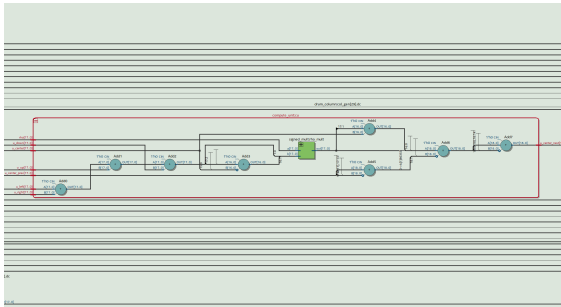


Figure 24: RTL View of compute unit.

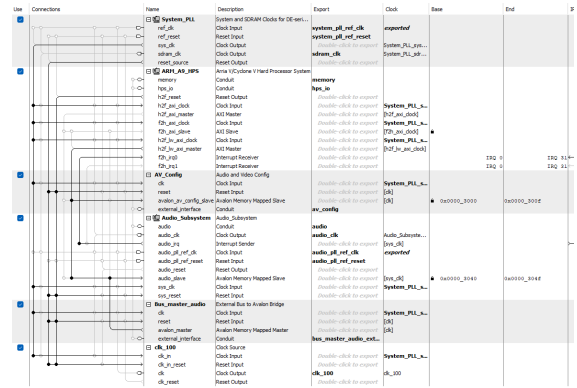


Figure 23: Quartus overview of devices on the Avalon Bus.

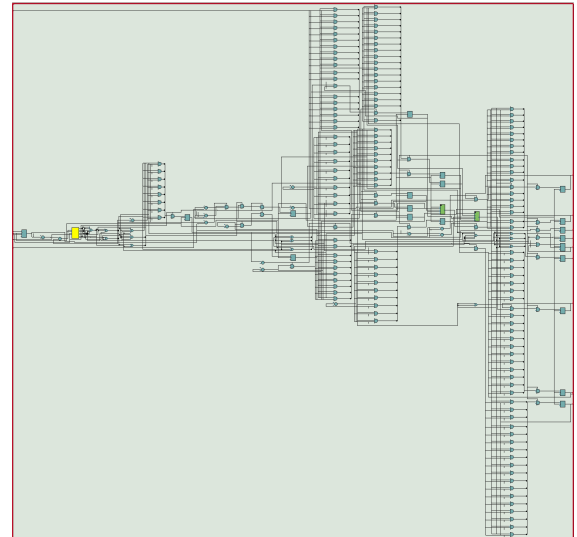


Figure 25: RTL View of column control path.

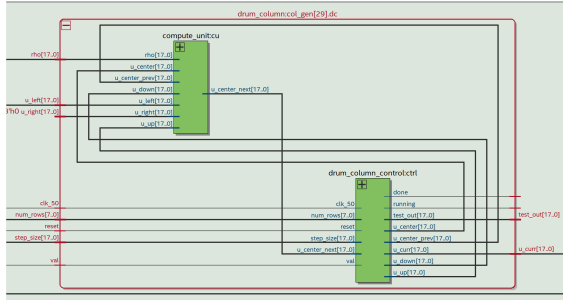


Figure 26: RTL View of column datapath.

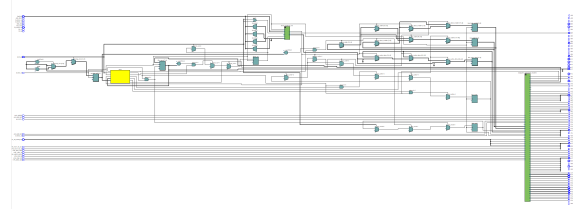


Figure 27: RTL View of drum top.

11 Comments

This lab helped us see the difficulties present in developing an accelerator for drum synthesis on the DE1-SoC and develop intuition of developing complex state machines to do iterative calculations with time constraints. We were able to understand better how to manage both parallelism and iterative processes within RTL as our columns and rows were developed through these different approaches. We were able to take a partial differential equation, discretize it and optimize it for implementation onto the FPGA, then performing active debugging and optimizations in order to try to get as many drum nodes as possible.

Additionally, this lab had us encounter yet another case of overflow, which allowed us to develop skills in diagnosis and treatment of conceptual bugs in RTL, which may be difficult to detect as it does not result in a compiler error, so can't be detected easily.

This lab also taught us much about frequencies and frequency analysis. After several discussions in lab with Bruce and Hunter, we have come away with a deeper understanding of how the fundamental frequency changes with size in a membrane. We also explored why only some harmonics appeared in the FFT analysis of our synthesized drum, and what effects the initial conditions may have had on the frequency content.

12 AI Usage

We used artificial intelligence at several instances throughout this project and primarily served as a supplementary tool for clarification, debugging, and writing refinement. While we were developing our design, AI was occasionally used to interpret compiler and synthesis error messages thrown at us by ModelSim or Quartus, helping to identify syntax issues, resolve signal width mismatches, and diagnose incorrect logic behavior in both simulation and hardware testing.

We used AI also as a conceptual aid when reviewing hardware and system-level design decisions. For instance, it helped clarify for us some aspects of Verilog implementation, finite state machine behavior (really important this lab), memory organization, and the interaction between different modules when unexpected behavior became apparent in our testing or simulations. When we worked outside of scheduled lab hours, AI acted as an alternate resource to reinforce our understanding of course concepts and assist in interpreting lab requirements especially when course staff were not available.

Additionally, AI was used to improve the preparation of this report in terms of clarity and accuracy. Its role was restricted to improving clarity by refining grammar, sentence structure, and overall flow to allow better readability for the audience. All the technical content, design decisions, and implementation details were developed independently by our team, while AI was used only to enhance readability and presentation.

Prompt Interaction Log

Time	Prompt Summary	Edits Suggested	Insertions	Deletions	Modifications
09:10 AM	User asked for guidance on scaling a 1D FSM for drum column writing to a 30×30 2D drum model	6	5	2	7
09:18 AM	Assistant proposed restructuring FSM with row/column counters, wait states, and 2D stencil window	8	7	3	9
09:26 AM	User requested a structured prompt log including edits and code change metrics in LaTeX format	2	3	0	2

Notes:

- Edits suggested correspond to conceptual or structural improvements to the FSM design.
- Insertions represent newly introduced states, counters, or data paths.
- Deletions reflect removal or replacement of 1D-specific logic.
- Modifications capture refinements to existing FSM behavior and memory handling.

References

- [1] V. H. Adams and B. Land, “Multiprocessor Drum Synthesis,” ECE 5760 Lab Hand-out, Cornell University, Spring 2026. https://vanhunteradams.com/DE1/Drum/Drum_synthesis.html
- [2] S. A. Van Duyne and J. O. Smith, “Physical Modeling with the 2-D Digital Waveguide Mesh,” *Proceedings of the International Computer Music Conference*, Tokyo, 1993.
- [3] Elsevier, “Nyquist Rate,” *ScienceDirect Topics*, 2026. Available: <https://www.sciencedirect.com/topics/engineering/nyquist-rate>. Accessed: March 27, 2026.

12.1 Program Listings

```
1
2
3 module DE1_SoC_Computer (
4     //////////////////////////////////////
5     // FPGA Pins
6     //////////////////////////////////////
7
8     // Clock pins
9     CLOCK_50 ,
10    CLOCK2_50 ,
11    CLOCK3_50 ,
12    CLOCK4_50 ,
13
14    // ADC
15    ADC_CS_N ,
16    ADC_DIN ,
17    ADC_DOUT ,
18    ADC_SCLK ,
19
20    // Audio
21    AUD_ADCDAT ,
22    AUD_ADCLRCK ,
23    AUD_BCLK ,
24    AUD_DACDAT ,
25    AUD_DACLCK ,
26    AUD_XCK ,
27
28    // SDRAM
29    DRAM_ADDR ,
30    DRAM_BA ,
31    DRAM_CAS_N ,
32    DRAM_CKE ,
33    DRAM_CLK ,
34    DRAM_CS_N ,
35    DRAM_DQ ,
36    DRAM_LDQM ,
37    DRAM_RAS_N ,
38    DRAM_UDQM ,
39    DRAM_WE_N ,
40
41    // I2C Bus for Configuration of the Audio and Video-In Chips
42    FPGA_I2C_SCLK ,
43    FPGA_I2C_SDAT ,
44
45    // 40-Pin Headers
46    GPIO_0 ,
```

```

47     GPIO_1 ,
48
49     // Seven Segment Displays
50     HEX0 ,
51     HEX1 ,
52     HEX2 ,
53     HEX3 ,
54     HEX4 ,
55     HEX5 ,
56
57     // IR
58     IRDA_RXD ,
59     IRDA_TXD ,
60
61     // Pushbuttons
62     KEY ,
63
64     // LEDs
65     LEDR ,
66
67     // PS2 Ports
68     PS2_CLK ,
69     PS2_DAT ,
70
71     PS2_CLK2 ,
72     PS2_DAT2 ,
73
74     // Slider Switches
75     SW ,
76
77     // Video-In
78     TD_CLK27 ,
79     TD_DATA ,
80     TD_HS ,
81     TD_RESET_N ,
82     TD_VS ,
83
84     // VGA
85     VGA_B ,
86     VGA_BLANK_N ,
87     VGA_CLK ,
88     VGA_G ,
89     VGA_HS ,
90     VGA_R ,
91     VGA_SYNC_N ,
92     VGA_VS ,
93

```

```

94  //////////////////////////////////////////
95  // HPS Pins
96  //////////////////////////////////////////
97
98  // DDR3 SDRAM
99  HPS_DDR3_ADDR ,
100 HPS_DDR3_BA ,
101 HPS_DDR3_CAS_N ,
102 HPS_DDR3_CKE ,
103 HPS_DDR3_CK_N ,
104 HPS_DDR3_CK_P ,
105 HPS_DDR3_CS_N ,
106 HPS_DDR3_DM ,
107 HPS_DDR3_DQ ,
108 HPS_DDR3_DQS_N ,
109 HPS_DDR3_DQS_P ,
110 HPS_DDR3_ODT ,
111 HPS_DDR3_RAS_N ,
112 HPS_DDR3_RESET_N ,
113 HPS_DDR3_RZQ ,
114 HPS_DDR3_WE_N ,
115
116 // Ethernet
117 HPS_ENET_GTX_CLK ,
118 HPS_ENET_INT_N ,
119 HPS_ENET_MDC ,
120 HPS_ENET_MDIO ,
121 HPS_ENET_RX_CLK ,
122 HPS_ENET_RX_DATA ,
123 HPS_ENET_RX_DV ,
124 HPS_ENET_TX_DATA ,
125 HPS_ENET_TX_EN ,
126
127 // Flash
128 HPS_FLASH_DATA ,
129 HPS_FLASH_DCLK ,
130 HPS_FLASH_NCS0 ,
131
132 // Accelerometer
133 HPS_GSENSOR_INT ,
134
135 // General Purpose I/O
136 HPS_GPIO ,
137
138 // I2C
139 HPS_I2C_CONTROL ,
140 HPS_I2C1_SCLK ,

```

```

141     HPS_I2C1_SDAT ,
142     HPS_I2C2_SCLK ,
143     HPS_I2C2_SDAT ,
144
145     // Pushbutton
146     HPS_KEY ,
147
148     // LED
149     HPS_LED ,
150
151     // SD Card
152     HPS_SD_CLK ,
153     HPS_SD_CMD ,
154     HPS_SD_DATA ,
155
156     // SPI
157     HPS_SPIM_CLK ,
158     HPS_SPIM_MISO ,
159     HPS_SPIM_MOSI ,
160     HPS_SPIM_SS ,
161
162     // UART
163     HPS_UART_RX ,
164     HPS_UART_TX ,
165
166     // USB
167     HPS_CONV_USB_N ,
168     HPS_USB_CLKOUT ,
169     HPS_USB_DATA ,
170     HPS_USB_DIR ,
171     HPS_USB_NXT ,
172     HPS_USB_STP
173 );
174
175 //=====
176 //  PARAMETER declarations
177 //=====
178
179
180 //=====
181 //  PORT declarations
182 //=====
183
184 //////////////////////////////////////
185 //  FPGA Pins
186 //////////////////////////////////////
187

```

```

188 // Clock pins
189 input          CLOCK_50;
190 input          CLOCK2_50;
191 input          CLOCK3_50;
192 input          CLOCK4_50;
193
194 // ADC
195 inout          ADC_CS_N;
196 output         ADC_DIN;
197 input          ADC_DOUT;
198 output         ADC_SCLK;
199
200 // Audio
201 input          AUD_ADCDAT;
202 inout          AUD_ADCLRCK;
203 inout          AUD_BCLK;
204 output         AUD_DACDAT;
205 inout          AUD_DACLCK;
206 output         AUD_XCK;
207
208 // SDRAM
209 output         [12: 0] DRAM_ADDR;
210 output         [ 1: 0] DRAM_BA;
211 output         DRAM_CAS_N;
212 output         DRAM_CKE;
213 output         DRAM_CLK;
214 output         DRAM_CS_N;
215 inout          [15: 0] DRAM_DQ;
216 output         DRAM_LDQM;
217 output         DRAM_RAS_N;
218 output         DRAM_UDQM;
219 output         DRAM_WE_N;
220
221 // I2C Bus for Configuration of the Audio and Video-In Chips
222 output         FPGA_I2C_SCLK;
223 inout          FPGA_I2C_SDAT;
224
225 // 40-pin headers
226 inout          [35: 0] GPIO_0;
227 inout          [35: 0] GPIO_1;
228
229 // Seven Segment Displays
230 output         [ 6: 0] HEX0;
231 output         [ 6: 0] HEX1;
232 output         [ 6: 0] HEX2;
233 output         [ 6: 0] HEX3;
234 output         [ 6: 0] HEX4;

```

```

235 output          [ 6: 0] HEX5;
236
237 // IR
238 input            IRDA_RXD;
239 output          IRDA_TXD;
240
241 // Pushbuttons
242 input            [ 3: 0] KEY;
243
244 // LEDs
245 output          [ 9: 0] LEDR;
246
247 // PS2 Ports
248 inout           PS2_CLK;
249 inout           PS2_DAT;
250
251 inout           PS2_CLK2;
252 inout           PS2_DAT2;
253
254 // Slider Switches
255 input            [ 9: 0] SW;
256
257 // Video-In
258 input            TD_CLK27;
259 input            [ 7: 0] TD_DATA;
260 input            TD_HS;
261 output          TD_RESET_N;
262 input            TD_VS;
263
264 // VGA
265 output          [ 7: 0] VGA_B;
266 output          VGA_BLANK_N;
267 output          VGA_CLK;
268 output          [ 7: 0] VGA_G;
269 output          VGA_HS;
270 output          [ 7: 0] VGA_R;
271 output          VGA_SYNC_N;
272 output          VGA_VS;
273
274
275
276 ////////////////////////////////////////////////////
277 // HPS Pins
278 ////////////////////////////////////////////////////
279
280 // DDR3 SDRAM
281 output          [14: 0] HPS_DDR3_ADDR;

```



```

282 output          [ 2: 0]  HPS_DDR3_BA;
283 output          HPS_DDR3_CAS_N;
284 output          HPS_DDR3_CKE;
285 output          HPS_DDR3_CK_N;
286 output          HPS_DDR3_CK_P;
287 output          HPS_DDR3_CS_N;
288 output          [ 3: 0] HPS_DDR3_DM;
289 inout           [31: 0] HPS_DDR3_DQ;
290 inout           [ 3: 0] HPS_DDR3_DQS_N;
291 inout           [ 3: 0] HPS_DDR3_DQS_P;
292 output          HPS_DDR3_ODT;
293 output          HPS_DDR3_RAS_N;
294 output          HPS_DDR3_RESET_N;
295 input           HPS_DDR3_RZQ;
296 output          HPS_DDR3_WE_N;
297
298 // Ethernet
299 output          HPS_ENET_GTX_CLK;
300 inout           HPS_ENET_INT_N;
301 output          HPS_ENET_MDC;
302 inout           HPS_ENET_MDIO;
303 input           HPS_ENET_RX_CLK;
304 input           [ 3: 0] HPS_ENET_RX_DATA;
305 input           HPS_ENET_RX_DV;
306 output          [ 3: 0] HPS_ENET_TX_DATA;
307 output          HPS_ENET_TX_EN;
308
309 // Flash
310 inout           [ 3: 0] HPS_FLASH_DATA;
311 output          HPS_FLASH_DCLK;
312 output          HPS_FLASH_NCS0;
313
314 // Accelerometer
315 inout           HPS_GSENSOR_INT;
316
317 // General Purpose I/O
318 inout           [ 1: 0] HPS_GPIO;
319
320 // I2C
321 inout           HPS_I2C_CONTROL;
322 inout           HPS_I2C1_SCLK;
323 inout           HPS_I2C1_SDAT;
324 inout           HPS_I2C2_SCLK;
325 inout           HPS_I2C2_SDAT;
326
327 // Pushbutton
328 inout           HPS_KEY;

```

```

329
330 // LED
331 inout                HPS_LED;
332
333 // SD Card
334 output                HPS_SD_CLK;
335 inout                HPS_SD_CMD;
336 inout                [ 3: 0] HPS_SD_DATA;
337
338 // SPI
339 output                HPS_SPIM_CLK;
340 input                HPS_SPIM_MISO;
341 output                HPS_SPIM_MOSI;
342 inout                HPS_SPIM_SS;
343
344 // UART
345 input                HPS_UART_RX;
346 output                HPS_UART_TX;
347
348 // USB
349 inout                HPS_CONV_USB_N;
350 input                HPS_USB_CLKOUT;
351 inout                [ 7: 0] HPS_USB_DATA;
352 input                HPS_USB_DIR;
353 input                HPS_USB_NXT;
354 output                HPS_USB_STP;
355
356 //=====
357 // REG/WIRE declarations
358 //=====
359
360 wire                [15: 0] hex3_hex0;
361 wire                clk_100;
362 //wire                [15: 0] hex5_hex4;
363
364 //assign HEX0 = ~hex3_hex0[ 6: 0]; // hex3_hex0[ 6: 0];
365 //assign HEX1 = ~hex3_hex0[14: 8];
366 //assign HEX2 = ~hex3_hex0[22:16];
367 //assign HEX3 = ~hex3_hex0[30:24];
368 assign HEX4 = 7'b1111111;
369 assign HEX5 = 7'b1111111;
370 assign HEX3 = 7'b1111111;
371 assign HEX2 = 7'b1111111;
372 assign HEX1 = 7'b1111111;
373 assign HEX0 = 7'b1111111;
374
375 //=====

```

```

376 // Bus controller for AVALON bus-master
377 //=====
378 // computes DDS for sine wave and fills audio FIFO
379
380 reg [31:0] bus_addr ; // Avalon address
381 // see
382 // ftp://ftp.altera.com/up/pub/Altera_Material/15.1/
    University_Program_IP_Cores/Audio_Video/Audio.pdf
383 // for addresses
384 wire [31:0] audio_base_address = 32'h00003040 ; // Avalon address
385 wire [31:0] audio_fifo_address = 32'h00003044 ; // Avalon address
    +4 offset
386 wire [31:0] audio_left_address = 32'h00003048 ; // Avalon address
    +8
387 wire [31:0] audio_right_address = 32'h0000304c ; // Avalon address
    +12
388 reg [3:0] bus_byte_enable ; // four bit byte read/write mask
389 reg bus_read ; // high when requesting data
390 reg bus_write ; // high when writing data
391 reg [31:0] bus_write_data ; // data to send to Avalon bus
392 wire bus_ack ; // Avalon bus raises this when done
393 wire [31:0] bus_read_data ; // data from Avalon bus
394 reg [30:0] timer ;
395 reg [3:0] state ;
396 wire state_clock ;
397
398 // current free words in audio interface
399 reg [7:0] fifo_space ;
400 // debug check of space
401 assign LEDR = fifo_space ;
402
403 // use 4-byte-wide bus-master
404 //assign bus_byte_enable = 4'b1111;
405
406 // ---- Configuration from switches ----
407 // SW[4:0]: number of rows (0 = default 30, otherwise clamped 4-30)
408 wire [7:0] cfg_num_rows;
409 //assign cfg_num_rows = (SW[4:0] == 5'd0) ? 6'd30 :
410 //                      (SW[4:0] < 5'd4) ? 6'd4 :
411 //                      (SW[4:0] > 5'd30) ? 6'd30 : {1'b0, SW
    [4:0]};
412
413 assign cfg_num_rows = SW[7:0];
414
415 // SW[7:5]: rho gain preset (all below stability limit of 1/4)
416 // 000 = default 1/16 (matches original hardcoded value)
417 reg signed [17:0] cfg_rho;

```

```

418 //always @(*) begin
419 //    case (SW[7:5])
420 //        3'd0: cfg_rho = 18'sd8192;    // 1/16 (DEFAULT - standard
           drum)
421 //        3'd1: cfg_rho = 18'sd2048;    // 1/64 (very deep bass)
422 //        3'd2: cfg_rho = 18'sd4096;    // 1/32 (deep drum)
423 //        3'd3: cfg_rho = 18'sd6144;    // 3/64 (low tom)
424 //        3'd4: cfg_rho = 18'sd10240;   // 5/64 (brighter)
425 //        3'd5: cfg_rho = 18'sd12288;   // 3/32 (high tom)
426 //        3'd6: cfg_rho = 18'sd14336;   // 7/64 (snare-like)
427 //        3'd7: cfg_rho = 18'sd16384;   // 1/8 (tight, max safe)
428 //        default: cfg_rho = 18'sd8192;
429 //    endcase
430 //end
431
432 always @(*) begin
433     cfg_rho <= 18'sd8192;
434 end
435
436 // SW[9:8]: initial amplitude (overflow-safe for Q1.17 with 30-col
           pyramid)
437 // 00 = default 512 (matches original hardcoded value)
438 reg signed [17:0] cfg_base_step;
439 always @(*) begin
440     case (SW[9:8])
441         2'd0: cfg_base_step = 18'sd128;    // DEFAULT - full (peak
           ~0.88)
442         2'd1: cfg_base_step = 18'sd64;     // moderate
443         2'd2: cfg_base_step = 18'sd32;     // quiet
444         2'd3: cfg_base_step = 18'sd16;     // very quiet
445         default: cfg_base_step = 18'sd512;
446     endcase
447 end
448
449 // KEY[1] re-strikes the drum (active low)
450 wire drum_reset = ~KEY[0] | ~KEY[1];
451
452 // Drum signals
453 reg        drum_val ;
454 wire       drum_rdy ;
455
456 //assign GPIO_0[1] = drum_rdy; // for timing
457
458 wire signed [17:0] drum_amplitude ;
459
460 drum_top drum_inst (
461     .clk(CLOCK_50),

```

```

462     .reset(drum_reset),
463     .val(drum_val),
464     .num_rows(cfg_num_rows),
465     .rho(cfg_rho),
466     .base_step(cfg_base_step),
467     .rdy(drum_rdy),
468     .amplitude(drum_amplitude),
469     .running(GPIO_0[1])
470 );
471
472 wire [31:0] audio_out_32 = {drum_amplitude[17:2], 16'd0};
473
474 // get some signals exposed
475 // connect bus master signals to i/o for probes
476 //assign GPIO_0[0] = bus_write ;
477 //assign GPIO_0[1] = bus_read ;
478 //assign GPIO_0[2] = bus_ack ;
479 //assign GPIO_0[3] = ??? ;
480
481 // Startup holdoff: wait for drum to finish INNIT (30+ cycles)
    before
482 // the bus master starts sending val. Prevents deadlock after reset.
483 reg [6:0] startup_cnt;
484
485 always @(posedge CLOCK_50) begin //CLOCK_50
486
487     // reset state machine and read/write controls
488     if (~KEY[0]) begin
489         state <= 0 ;
490         bus_read <= 0 ;
491         bus_write <= 0 ;
492         timer <= 0;
493         drum_val <= 0;
494         startup_cnt <= 7'd0;
495     end
496     else begin
497         timer <= timer + 1;
498         if (startup_cnt < 7'd100)
499             startup_cnt <= startup_cnt + 7'd1;
500     end
501
502     // set up read FIFO available space (only after startup holdoff)
503     if (state==4'd0 && startup_cnt >= 7'd100) begin
504         bus_addr <= audio_fifo_address ;
505         bus_read <= 1'b1 ;
506         bus_byte_enable <= 4'b1111;
507         state <= 4'd1 ; // wait for read ACK

```

```

508     end
509
510     // wait for read ACK and read the fifo available
511     if (state==4'd1 && bus_ack==1) begin
512         state <= 4'd2 ;
513         fifo_space <= (bus_read_data>>24) ;
514         bus_read <= 1'b0 ;
515     end
516
517     // When there is room in the FIFO
518     if (state==4'd2 && fifo_space>8'd2) begin
519         drum_val <= 1'b1 ;
520         state <= 4'd6;
521     end
522     else if (state==4'd2 && fifo_space<=8'd2) begin
523         state <= 4'd0 ;
524     end
525
526     // Wait for drum to be ready
527     if (state==4'd6) begin
528         drum_val <= 1'b0 ;
529         if (drum_rdy) begin
530             bus_write_data <= audio_out_32 ;
531             bus_addr <= audio_left_address ;
532             bus_byte_enable <= 4'b1111;
533             bus_write <= 1'b1 ;
534             state <= 4'd3 ;
535         end
536     end
537
538     // detect bus-transaction-complete ACK
539     if (state==4'd3 && bus_ack==1) begin
540         state <= 4'd4 ;
541         bus_write <= 0;
542     end
543
544     // -- now the right channel
545     if (state==4'd4) begin
546         state <= 4'd5;
547         bus_write_data <= audio_out_32 ;
548         bus_addr <= audio_right_address ;
549         bus_write <= 1'b1 ;
550     end
551
552     // detect bus-transaction-complete ACK
553     if (state==4'd5 && bus_ack==1) begin
554         state <= 4'd0 ;

```

```

555         bus_write <= 0;
556     end
557
558 end // always @(posedge CLOCK_50)
559
560
561 //=====
562 // Structural coding
563 //=====
564
565 Computer_System The_System (
566     //////////////////////////////////////
567     // FPGA Side
568     //////////////////////////////////////
569
570     // Global signals
571     .system_pll_ref_clk_clk          (CLOCK_50),
572     .system_pll_ref_reset_reset      (1'b0),
573     .sdram_clk_clk                   (state_clock),
574     .clk_100_clk                     (clk_100),
575
576     // AV Config
577     .av_config_SCLK                  (FPGA_I2C_SCLK),
578     .av_config_SDAT                  (FPGA_I2C_SDAT),
579
580     // Audio Subsystem
581     .audio_pll_ref_clk_clk           (CLOCK3_50),
582     .audio_pll_ref_reset_reset       (1'b0),
583     .audio_clk_clk                   (AUD_XCK),
584     .audio_ADCDAT                    (AUD_ADCDAT),
585     .audio_ADCLRCK                   (AUD_ADCLRCK),
586     .audio_BCLK                      (AUD_BCLK),
587     .audio_DACDAT                    (AUD_DACDAT),
588     .audio_DACLCK                    (AUD_DACLCK),
589
590     // bus-master state machine interface
591     .bus_master_audio_external_interface_address (bus_addr),
592     .bus_master_audio_external_interface_byte_enable (
593         bus_byte_enable),
594     .bus_master_audio_external_interface_read      (bus_read),
595     .bus_master_audio_external_interface_write     (bus_write),
596     .bus_master_audio_external_interface_write_data (bus_write_data
597     ),
598     .bus_master_audio_external_interface_acknowledge (bus_ack),
599     .bus_master_audio_external_interface_read_data  (bus_read_data)
600     ,

```

```

599
600 ////////////////////////////////////////////////////
601 // HPS Side
602 ////////////////////////////////////////////////////
603 // DDR3 SDRAM
604 .memory_mem_a          (HPS_DDR3_ADDR),
605 .memory_mem_ba         (HPS_DDR3_BA),
606 .memory_mem_ck         (HPS_DDR3_CK_P),
607 .memory_mem_ck_n       (HPS_DDR3_CK_N),
608 .memory_mem_cke        (HPS_DDR3_CKE),
609 .memory_mem_cs_n       (HPS_DDR3_CS_N),
610 .memory_mem_ras_n      (HPS_DDR3_RAS_N),
611 .memory_mem_cas_n      (HPS_DDR3_CAS_N),
612 .memory_mem_we_n       (HPS_DDR3_WE_N),
613 .memory_mem_reset_n    (HPS_DDR3_RESET_N),
614 .memory_mem_dq         (HPS_DDR3_DQ),
615 .memory_mem_dqs        (HPS_DDR3_DQS_P),
616 .memory_mem_dqs_n      (HPS_DDR3_DQS_N),
617 .memory_mem_odt        (HPS_DDR3_ODT),
618 .memory_mem_dm         (HPS_DDR3_DM),
619 .memory_oct_rzqin      (HPS_DDR3_RZQ),
620
621 // Ethernet
622 .hps_io_hps_io_gpio_inst_GPIO35 (HPS_ENET_INT_N),
623 .hps_io_hps_io_emac1_inst_TX_CLK (HPS_ENET_GTX_CLK),
624 .hps_io_hps_io_emac1_inst_TXD0 (HPS_ENET_TX_DATA[0]),
625 .hps_io_hps_io_emac1_inst_TXD1 (HPS_ENET_TX_DATA[1]),
626 .hps_io_hps_io_emac1_inst_TXD2 (HPS_ENET_TX_DATA[2]),
627 .hps_io_hps_io_emac1_inst_TXD3 (HPS_ENET_TX_DATA[3]),
628 .hps_io_hps_io_emac1_inst_RXD0 (HPS_ENET_RX_DATA[0]),
629 .hps_io_hps_io_emac1_inst_MDIO (HPS_ENET_MDIO),
630 .hps_io_hps_io_emac1_inst_MDC (HPS_ENET_MDC),
631 .hps_io_hps_io_emac1_inst_RX_CTL (HPS_ENET_RX_DV),
632 .hps_io_hps_io_emac1_inst_TX_CTL (HPS_ENET_TX_EN),
633 .hps_io_hps_io_emac1_inst_RX_CLK (HPS_ENET_RX_CLK),
634 .hps_io_hps_io_emac1_inst_RXD1 (HPS_ENET_RX_DATA[1]),
635 .hps_io_hps_io_emac1_inst_RXD2 (HPS_ENET_RX_DATA[2]),
636 .hps_io_hps_io_emac1_inst_RXD3 (HPS_ENET_RX_DATA[3]),
637
638 // Flash
639 .hps_io_hps_io_qspi_inst_I00 (HPS_FLASH_DATA[0]),
640 .hps_io_hps_io_qspi_inst_I01 (HPS_FLASH_DATA[1]),
641 .hps_io_hps_io_qspi_inst_I02 (HPS_FLASH_DATA[2]),
642 .hps_io_hps_io_qspi_inst_I03 (HPS_FLASH_DATA[3]),
643 .hps_io_hps_io_qspi_inst_SS0 (HPS_FLASH_NCS0),
644 .hps_io_hps_io_qspi_inst_CLK (HPS_FLASH_DCLK),
645

```



```

646 // Accelerometer
647 .hps_io_hps_io_gpio_inst_GPIO61 (HPS_GSENSOR_INT),
648
649 // .adc_sclk (ADC_SCLK),
650 // .adc_cs_n (ADC_CS_N),
651 // .adc_dout (ADC_DOUT),
652 // .adc_din (ADC_DIN),
653
654 // General Purpose I/O
655 .hps_io_hps_io_gpio_inst_GPIO40 (HPS_GPIO[0]),
656 .hps_io_hps_io_gpio_inst_GPIO41 (HPS_GPIO[1]),
657
658 // I2C
659 .hps_io_hps_io_gpio_inst_GPIO48 (HPS_I2C_CONTROL),
660 .hps_io_hps_io_i2c0_inst_SDA (HPS_I2C1_SDAT),
661 .hps_io_hps_io_i2c0_inst_SCL (HPS_I2C1_SCLK),
662 .hps_io_hps_io_i2c1_inst_SDA (HPS_I2C2_SDAT),
663 .hps_io_hps_io_i2c1_inst_SCL (HPS_I2C2_SCLK),
664
665 // Pushbutton
666 .hps_io_hps_io_gpio_inst_GPIO54 (HPS_KEY),
667
668 // LED
669 .hps_io_hps_io_gpio_inst_GPIO53 (HPS_LED),
670
671 // SD Card
672 .hps_io_hps_io_sdio_inst_CMD (HPS_SD_CMD),
673 .hps_io_hps_io_sdio_inst_D0 (HPS_SD_DATA[0]),
674 .hps_io_hps_io_sdio_inst_D1 (HPS_SD_DATA[1]),
675 .hps_io_hps_io_sdio_inst_CLK (HPS_SD_CLK),
676 .hps_io_hps_io_sdio_inst_D2 (HPS_SD_DATA[2]),
677 .hps_io_hps_io_sdio_inst_D3 (HPS_SD_DATA[3]),
678
679 // SPI
680 .hps_io_hps_io_spim1_inst_CLK (HPS_SPIM_CLK),
681 .hps_io_hps_io_spim1_inst_MOSI (HPS_SPIM_MOSI),
682 .hps_io_hps_io_spim1_inst_MISO (HPS_SPIM_MISO),
683 .hps_io_hps_io_spim1_inst_SS0 (HPS_SPIM_SS),
684
685 // UART
686 .hps_io_hps_io_uart0_inst_RX (HPS_UART_RX),
687 .hps_io_hps_io_uart0_inst_TX (HPS_UART_TX),
688
689 // USB
690 .hps_io_hps_io_gpio_inst_GPIO09 (HPS_CONV_USB_N),
691 .hps_io_hps_io_usb1_inst_D0 (HPS_USB_DATA[0]),
692 .hps_io_hps_io_usb1_inst_D1 (HPS_USB_DATA[1]),

```

```

693     .hps_io_hps_io_usb1_inst_D2      (HPS_USB_DATA[2]),
694     .hps_io_hps_io_usb1_inst_D3      (HPS_USB_DATA[3]),
695     .hps_io_hps_io_usb1_inst_D4      (HPS_USB_DATA[4]),
696     .hps_io_hps_io_usb1_inst_D5      (HPS_USB_DATA[5]),
697     .hps_io_hps_io_usb1_inst_D6      (HPS_USB_DATA[6]),
698     .hps_io_hps_io_usb1_inst_D7      (HPS_USB_DATA[7]),
699     .hps_io_hps_io_usb1_inst_CLK      (HPS_USB_CLKOUT),
700     .hps_io_hps_io_usb1_inst_STP      (HPS_USB_STP),
701     .hps_io_hps_io_usb1_inst_DIR      (HPS_USB_DIR),
702     .hps_io_hps_io_usb1_inst_NXT      (HPS_USB_NXT)
703 );
704
705
706 endmodule
707
708
709 // Below is the drum code we wrote and verified on ModelSim and now
integrated into the Audio quartus project
710
711 /**
712  * signed_mult: 1.17 fixed-point multiplier
713  */
714 module signed_mult (
715     output signed [17:0] out,
716     input  signed [17:0] a,
717     input  signed [17:0] b
718 );
719     wire signed [35:0] mult_out;
720     assign mult_out = a * b;
721     assign out      = {mult_out[35], mult_out[34:17]}; // added the
mult_out[35] (AI helped debug this issue)
722 endmodule
723
724
725 module M10K_540_18#
726 (
727     // following the format from last lab using same ports
728     output reg [17:0] q,
729     input  [17:0] d,
730     input  [9:0]  write_address,
731     input  [9:0]  read_address,
732     input                we,
733     input                clk
734 );
735     // decreased M10K block size to closely fit size of columns
736     reg [17:0] mem [200:0] /* synthesis ramstyle = "no_rw_check,
M10K" */;

```

```

737     always @(posedge clk) begin
738         if (we)
739             mem[write_address] <= d;
740             q <= mem[read_address];
741     end
742 endmodule
743
744 /**
745  * this is our compute unit represent the 2D wave equation
746  */
747
748 // we had to make some adjustments to our original compute unit for
749 // the single wave node
750 // we had originally made some assumptions which caused our code to
751 // crash later on
752 module compute_unit
753 (
754     // we tried matching the naming conventions to the screenshots
755     // of modelsim in the webpage
756     input  signed [17:0] u_north,
757     input  signed [17:0] u_below,
758     input  signed [17:0] u_left,
759     input  signed [17:0] u_right,
760     input  signed [17:0] u_center,
761     input  signed [17:0] u_center_p,
762     input  signed [17:0] rho,           // configurable rho gain
763     output signed [17:0] u_center_next
764 );
765
766 // discrete 5-point Laplacian
767 wire signed [17:0] laplacian;
768 assign laplacian = u_left + u_right + u_north + u_below
769     - (u_center <<< 2);
770
771 // wave-speed-scaled Laplacian
772 wire signed [17:0] rho_times_laplacian;
773 signed_mult rho_mult
774 (
775     .a    (rho),
776     .b    (laplacian),
777     .out  (rho_times_laplacian)
778 );
779
780 wire signed [17:0] u_rho_two_center;
781 assign u_rho_two_center = (u_center <<< 1) +
782     rho_times_laplacian;

```

```

779 // Here we are damping the previous step term
780 wire signed [17:0] center_prev_damped;
781 assign center_prev_damped = u_center_p - (u_center_p >>> 13);
782
783 // this raws the next-step init_displacement before the self-
784 // damping occurs
785 wire signed [17:0] u_next_raw;
786 assign u_next_raw = u_rho_two_center - center_prev_damped
787 ;
788 // Apply light self-damping
789 assign u_center_next = u_next_raw - (u_next_raw >>> 13);
790
791 endmodule
792
793 /**
794 * drum_col_ctrl_system: Serial row update for one column
795 * num_rows is a runtime input (4-30)
796 */
797 module drum_col_ctrl_system
798 (
799     input wire clk_50,
800     input wire reset,
801     input wire val,
802     input wire [7:0] num_rows, // runtime row count
803     input wire signed [17:0] u_center_next,
804     input wire signed [17:0] step_size,
805
806     output reg signed [17:0] u_center_p,
807     output reg signed [17:0] u_center,
808     output reg signed [17:0] u_north,
809     output reg signed [17:0] u_below,
810     output wire signed [17:0] u_curr, // exposed center for
811         lateral coupling
812
813     // tried matching naming so our ModelSim simulations can be
814     // comparable to th webpage simulations
815     output reg [17:0] test_out,
816     output reg [2:0] STATE_out,
817     output reg [7:0] row_out,
818     output reg done,
819     output reg running
820 );
821
822 reg [2:0] STATE;

```

```

822 reg [7:0] row_counter;
823 reg write_enable;
824 reg signed [17:0] init_displacement;
825 reg [9:0] innit_filladdress;
826 // We had to add an extra wait STATE to our design
827 localparam INNIT = 3'd0,
828             WAIT = 3'd6,
829             SETUP0 = 3'd1,
830             SETUP1 = 3'd2,
831             PIPELINE_PRIME = 3'd3,
832             SWEEP = 3'd4,
833             FLUSH = 3'd5;
834
835 // these variables were introduced during lecture from
836 // psuedocode on how to start the FSM
837 // we added several STATES and logic to fsm from class
838 // Most of this code esp the mem logic was mentioned in class so
839 // we copied exactly what the professor had suggested through
840 // the psuedocode with some minor adjustments
841 // code directly from class
842 reg signed [17:0] M10k_n_write_data;
843 reg signed [17:0] M10k_nm1_write_data;
844 reg [9:0] M10k_n_write_data;
845 reg [9:0] M10k_nm1_write_addr;
846 reg [9:0] M10k_n_read_addr;
847 reg [9:0] M10k_nm1_read_addr;
848 wire signed [17:0] M10k_n_out;
849 wire signed [17:0] M10k_nm1_out;
850
851 // Latch num_rows on reset so it stays stable during a sweep
852 reg [7:0] latched_num_rows;
853
854 assign u_curr = u_center;
855
856 always @(posedge clk_50) begin
857     if (reset) begin
858         STATE <= INNIT;
859         M10k_n_write_data <= 10'd0;
860         M10k_nm1_write_addr <= 10'd0;
861         M10k_n_read_addr <= 10'd0;
862         M10k_nm1_read_addr <= 10'd0;
863         row_counter <= 8'd0;
864         innitfilladdress <= 10'd0;
865         u_north <= 18'sd0;
866         u_below <= 18'sd0;
867         u_center <= 18'sd0;
868         u_center_p <= 18'sd0;

```

```

866         write_enable      <= 1'b0;
867         init_displacement  <= 18'sd0;
868         test_out           <= 18'd0;
869         STATE_out          <= 3'd0;
870         row_out            <= 8'd0;
871         done               <= 1'b0;
872         latched_num_rows   <= num_rows;
873
874     end else begin
875         STATE_out          <= STATE;
876         row_out            <= row_counter;
877         done               <= 1'b0;
878
879         case (STATE)
880             // Set initial condition for every node in the mesh
881             // (Pyramid)
882             INNIT: begin
883                 write_enable      <= 1'b1;
884                 innitfilladdress <= innitfilladdress + 10'd1;
885                 row_counter      <= row_counter + 8'd1;
886                 M10k_n_write_data <= init_displacement;
887                 M10k_nm1_write_data <= init_displacement;
888
889                 M10k_n_write_data <= innitfilladdress;
890                 M10k_nm1_write_addr <= innitfilladdress;
891                 // ##### BEGIN AI-GENERATED CODE
892                 #####
893                 if (row_counter < (latched_num_rows >> 1))
894                     init_displacement <= init_displacement +
895                         step_size;
896                 else
897                     init_displacement <= init_displacement -
898                         step_size;
899
900                 if (row_counter == latched_num_rows - 8'd1)
901                     STATE <= WAIT;
902             end
903             // ##### END AI-GENERATED CODE
904             #####
905
906             // wait for valid signal (occurs at 48KHz)
907             WAIT: begin
908                 write_enable      <= 1'b0;
909                 if (val) STATE <= SETUP0;
910             end
911
912             // Toggle timing gpio, send read request to m10k

```

```

907     SETUP0: begin
908         running                <= 1;
909         write_enable           <= 1'b0;
910         row_counter            <= 8'd0;
911         M10k_n_read_addr       <= 10'd1;
912         M10k_nm1_read_addr     <= 10'd0;
913         STATE                  <= SETUP1;
914     end
915
916     // send second read request to M10K
917     SETUP1: begin
918         M10k_n_read_addr <= 10'd2;
919         M10k_nm1_read_addr <= 10'd1;
920         STATE            <= PIPELINE_PRIME;
921     end
922
923     // latch u_north and u_center_p for each row,
924     // iterate from here NUM_ROWS times
925     PIPELINE_PRIME: begin
926         u_center                <= 18'sd0;
927         u_below                 <= 18'sd0;
928         M10k_n_read_addr <= M10k_n_read_addr + 10'd1;
929         M10k_nm1_read_addr <= M10k_nm1_read_addr + 10'd1
930         ;
931         u_north                <= M10k_n_out;
932         u_center_p              <= M10k_nm1_out;
933         STATE                  <= SWEEP;
934     end
935
936     // compute amplitude for this row (node) and store
937     // in M10K. Increment row counter
938     SWEEP: begin
939         M10k_n_read_addr <= M10k_n_read_addr + 10'd1;
940         M10k_nm1_read_addr <= M10k_nm1_read_addr + 10'd1
941         ;
942         // ##### BEGIN AI-GENERATED CODE
943         // #####
944         u_north                <= (row_counter ==
945             latched_num_rows - 8'd2) ? 18'sd0 :
946             M10k_n_out;
947         u_center                <= u_north;
948         u_below                 <= (row_counter == 8'd0) ? 18'
949             sd0 : u_center;
950         u_center_p              <= M10k_nm1_out;
951         // ##### END AI-GENERATED CODE
952         // #####

```

```

945         write_enable          <= 1'b1;
946         M10k_n_write_data <= u_center_next;
947         M10k_nm1_write_data <= u_center;
948         M10k_n_write_data <= (row_counter == 8'd0) ? 10'
           d0 : M10k_n_write_data + 10'd1;
949         M10k_nm1_write_addr <= (row_counter == 8'd0) ?
           10'd0 : M10k_nm1_write_addr + 10'd1;
950
951         if (row_counter == (latched_num_rows >> 1))
952             test_out <= u_center_next;
953
954         row_counter <= row_counter + 8'd1;
955         STATE      <= (row_counter == latched_num_rows
           - 8'd1) ? FLUSH : SWEEP;
956     end
957
958     // Toggle timing gpio, send done signal, write final
row to memory
959     FLUSH: begin
960         write_enable      <= 1'b1;
961         M10k_n_write_data <= M10k_n_write_data + 10'd1;
962         M10k_nm1_write_addr <= M10k_nm1_write_addr + 10'
           d1;
963         M10k_n_write_data <= u_center_next;
964         M10k_nm1_write_data <= u_center;
965         done              <= 1'b1;
966         running           <= 1'b0;
967         STATE             <= WAIT;
968     end
969
970     default: STATE <= WAIT;
971 endcase
972 end
973 end
974
975 M10K_540_18 M10k_current
976 (
977     .q          (M10k_n_out),
978     .d          (M10k_n_write_data),
979     .write_address (M10k_n_write_data),
980     .read_address  (M10k_n_read_addr),
981     .we           (write_enable),
982     .clk          (clk_50)
983 );
984
985 M10K_540_18 M10k_prev
986 (

```



```

987         .q                (M10k_nm1_out),
988         .d                (M10k_nm1_write_data),
989         .write_address    (M10k_nm1_write_addr),
990         .read_address     (M10k_nm1_read_addr),
991         .we               (write_enable),
992         .clk              (clk_50)
993     );
994
995 endmodule
996
997 /**
998  * drum_column connects the FSM and the compute unit so they work
999  * together
1000 */
1001 module drum_column
1002 (
1003     input  wire          clk_50,
1004     input  wire          reset,
1005     input  wire          val,
1006     input  wire [7:0]    num_rows,
1007     input  wire signed [17:0] rho,
1008     input  wire signed [17:0] u_left,
1009     input  wire signed [17:0] u_right,
1010     input  wire signed [17:0] step_size,
1011     output wire signed [17:0] u_curr,
1012     output wire [17:0]    test_out,
1013     output wire          done
1014 );
1015
1016     wire signed [17:0] u_north, u_below, u_center, u_center_p,
1017         u_center_next;
1018
1019     compute_unit computeunit
1020     (
1021         .u_north      (u_north),
1022         .u_below      (u_below),
1023         .u_left       (u_left),
1024         .u_right      (u_right),
1025         .u_center     (u_center),
1026         .u_center_p   (u_center_p),
1027         .rho          (rho),
1028         .u_center_next (u_center_next)
1029     );
1030
1031     drum_col_ctrl_system ctrl
1032     (
1033         .clk_50      (clk_50),
1034         .reset       (reset),

```

```

1032         .val                (val),
1033         .num_rows            (num_rows),
1034         .step_size           (step_size),
1035         .u_center_next       (u_center_next),
1036         .u_north              (u_north),
1037         .u_below              (u_below),
1038         .u_center_p           (u_center_p),
1039         .u_center             (u_center),
1040         .u_curr               (u_curr),
1041         .test_out             (test_out),
1042         .done                 (done)
1043     );
1044
1045 endmodule
1046
1047 /**
1048  * drum_top is the 30-column drum with configurable rows, rho, and
1049   * amplitude
1050  */
1051 module drum_top # ( parameter [7:0] NUM_COLS = 128)
1052 (
1053     input  wire          clk,
1054     input  wire          reset,
1055     input  wire          val,
1056     input  wire [7:0]    num_rows,
1057     input  wire signed [17:0] rho,
1058     input  wire signed [17:0] base_step,
1059     output wire          rdy,
1060     output wire signed [17:0] amplitude
1061 );
1062
1063     wire [17:0] u_curr_bus    [NUM_COLS];
1064     wire [17:0] test_out_bus  [NUM_COLS];
1065     wire [NUM_COLS - 1:0]    done_bus;
1066
1067     // Center node amplitude (Col 15, Row tap)
1068     assign amplitude = test_out_bus[NUM_COLS >> 1];
1069     assign rdy       = done_bus[0];
1070
1071     // Pyramid initial condition: step_size triangle across columns
1072     // Adder chain instead of multipliers to avoid DSP usage
1073     wire signed [17:0] step_size [NUM_COLS];
1074     assign step_size[0] = 18'sd0;
1075     genvar c;
1076     // Our original generate STATEment was wrong so AI helped debug
1077     // our mistakes
1078     generate

```

```

1077     for (c = 1; c < NUM_COLS; c = c + 1) begin : step_gen
1078         // ##### BEGIN AI-GENERATED CODE
1079         #####
1080         assign step_size[c] = (c <= (NUM_COLS >> 1))
1081             ? step_size[c-1] + base_step
1082             : step_size[c-1] - base_step;
1083     end
1084     // ##### END AI-GENERATED CODE
1085     #####
1086
1087 endgenerate
1088
1089 // Column instantiations with lateral coupling (boundaries fixed
1090 // to 0)
1091 generate
1092     for (c = 0; c < NUM_COLS; c = c + 1) begin : col_gen
1093         drum_column dc (
1094             .clk_50(clk), .reset(reset), .val(val),
1095             .num_rows(num_rows),
1096             .rho(rho),
1097             // ##### BEGIN AI-GENERATED CODE
1098             #####
1099             .u_left (c == 0 ? 18'sd0 : u_curr_bus[c-1 < 0 ? 0 :
1100                 c-1]),
1101             .u_right(c == NUM_COLS - 1 ? 18'sd0 : u_curr_bus[c+1
1102                 > NUM_COLS - 1 ? NUM_COLS - 1 : c+1]),
1103             // ##### END AI-GENERATED CODE
1104             #####
1105             .step_size(step_size[c]),
1106             .u_curr(u_curr_bus[c]),
1107             .test_out(test_out_bus[c]),
1108             .done(done_bus[c])
1109         );
1110     end
1111 endgenerate
1112 endmodule

```

Listing 2: DE1_SoC_Computer.v